

DEV✓CORE

Clash, Burn, and Exploit:
Manipulate Filters to
Pwn kernelCTF

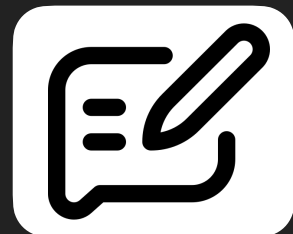
HexRabbit @ DEFCON 32

whoami

- HexRabbit (@h3xr4bb1t)
- Security researcher @ **DEV**CORE
- Specialized in binary exploitation



@h3xr4bb1t



<https://blog.hexrabbit.io>



h3xrabbit@gmail.com

What to present today?

- Introduce kernelCTF / nftables
- Three vulnerabilities found in nftables
- Stories behind my kernelCTF journey

kernelCTF

- Google VRP
- Accept Linux kernel LPE vulnerability
 - 0-day / 1-day
- Three target instances
 - LTS, COS, Mitigation
- \$\$\$ bounty



Google Bug Hunters

1. Exploit for the latest LTS instance

This instance uses the latest LTS (currently 6.1, soon 6.6) with [COS kernel config](#), but with `io_uring` and `nftables` (for 6.6) [disabled](#). Only the first submission is eligible per LTS kernel version, but we are upgrading the kernel version every 2-4 weeks on average.

Rewards

- Base reward: \$21.337
- Stability bonus (+\$10.000)
- Reduced attack surface bonus (+\$20.000)
- 0-day bonus (+\$20.000)



2. Mitigation bypass (on the mitigation instance)

The mitigation instance is upgraded far less frequently than the LTS instance (currently staying on 6.1.55), thus more 1-day vulnerabilities can be exploited. This way you have more opportunity to present your mitigation bypass techniques.

Only exploits which clearly bypass [our mitigations](#) are eligible (e.g. if a mitigation protects against UAF, but not against BoF exploit using a BoF vulnerability is not eligible).

As the current instance (`mitigation-v3-6.1.55`) only exploits with at least 70% reliability are eligible.

Reward

- \$21.000

3. Exploits for COS instances

These instances follow the live COS kernel config (which is also used in our infrastructure). `io_uring` and `nftables` are enabled here.

Only the first submission is eligible per COS version unless it is part of a security update, which happens every few weeks on average.

Reward

- \$21.000 if the exploit does not use user namespaces and `io_uring`
- \$10.500 if the exploit uses user namespaces or `io_uring`

Why nftables?

- Configs enabled in kernelCTF
- Multiple vulns discovered

Public kCTF VRP / kernelCTF responses

kernelCTF	kCTF VRP
netfilter: nf_tables: disallow rule removal from chain binding	CVE-2023-5197
netfilter: nf_tables: disallow rule removal from chain binding	CVE-2023-5197
netfilter: nf_tables: disallow rule removal from chain binding	CVE-2023-5197
net/sched: sch_hfsc: Ensure inner classes have fsc curve	CVE-2023-4623
netfilter: nf_tables: disallow rule removal from chain binding	CVE-2023-5197
netfilter: nf_tables: skip immediate deactivate in _PREPARE_ERROR	CVE-2023-4015
af_unix: Fix null-ptr-deref in unix_stream_sendpage().	CVE-2023-4622
netfilter: nf_tables: prevent OOB access in nft_byteorder_eval	CVE-2023-35001
net/sched: sch_hfsc: Ensure inner classes have fsc curve	CVE-2023-4623
af_unix: Fix null-ptr-deref in unix_stream_sendpage().	CVE-2023-4622
Merge tag 'nf-23-08-10' of git://git.kernel.org/pub/scm/linux/kernel/git/netfilter/ netfilter: nf_tables: remove busy mark and gc batch API netfilter: nft_set_hash: mark set element as dead when deleting from packet netfilter: nf_tables: adapt set backend to use GC transaction API netfilter: nf_tables: GC transaction API to avoid race with control plane netfilter: nf_tables: don't skip expired elements during walk	CVE-2023-4244
netfilter: nf_tables: disallow rule addition to bound chain via NFTA_RULE_CHAIN	CVE-2023-4147
net: sched: sch_qfq: Fix UAF in qfq_dequeue()	CVE-2023-4921
net: sched: sch_qfq: Fix UAF in qfq_dequeue()	CVE-2023-4921
netfilter: nf_tables: disallow rule addition to bound chain via NFTA_RULE_CHAIN	CVE-2023-4147
netfilter: nf_tables: disallow rule addition to bound chain via NFTA_RULE_CHAIN	CVE-2023-4147
netfilter: nf_tables: deactivate catchall elements in next generation	CVE-2023-4569
netfilter: nf_tables: skip bound chain on rule flush	CVE-2023-3777
netfilter: nf_tables: skip bound chain on rule flush	CVE-2023-3777

Intro to nftables

What is nftables?

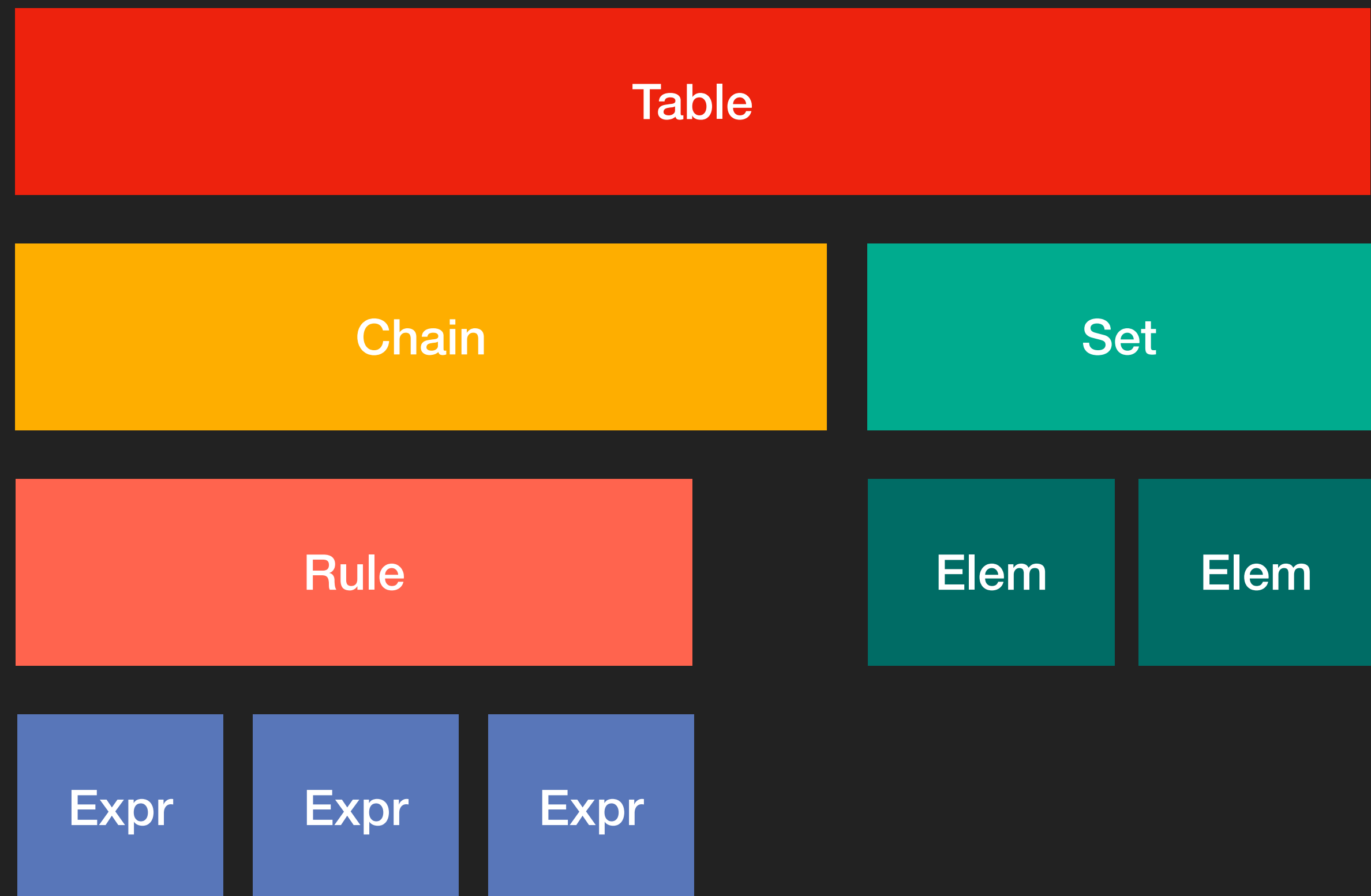
- Linux kernel packet filtering framework
- Replace legacy {ip/arp/eb}_tables
- Simple VM design
- New syntax for CLI interface

```
# iptables
iptables -P INPUT DROP
iptables -A INPUT -p tcp --dport 22 -j ACCEPT
iptables -A INPUT -i lo -j ACCEPT

# nftables
nft add table ip table0 {
    chain chain0 {
        type filter hook input priority 0; policy drop;
        tcp dport 22 accept
        iifname "lo" accept
    }
}
```

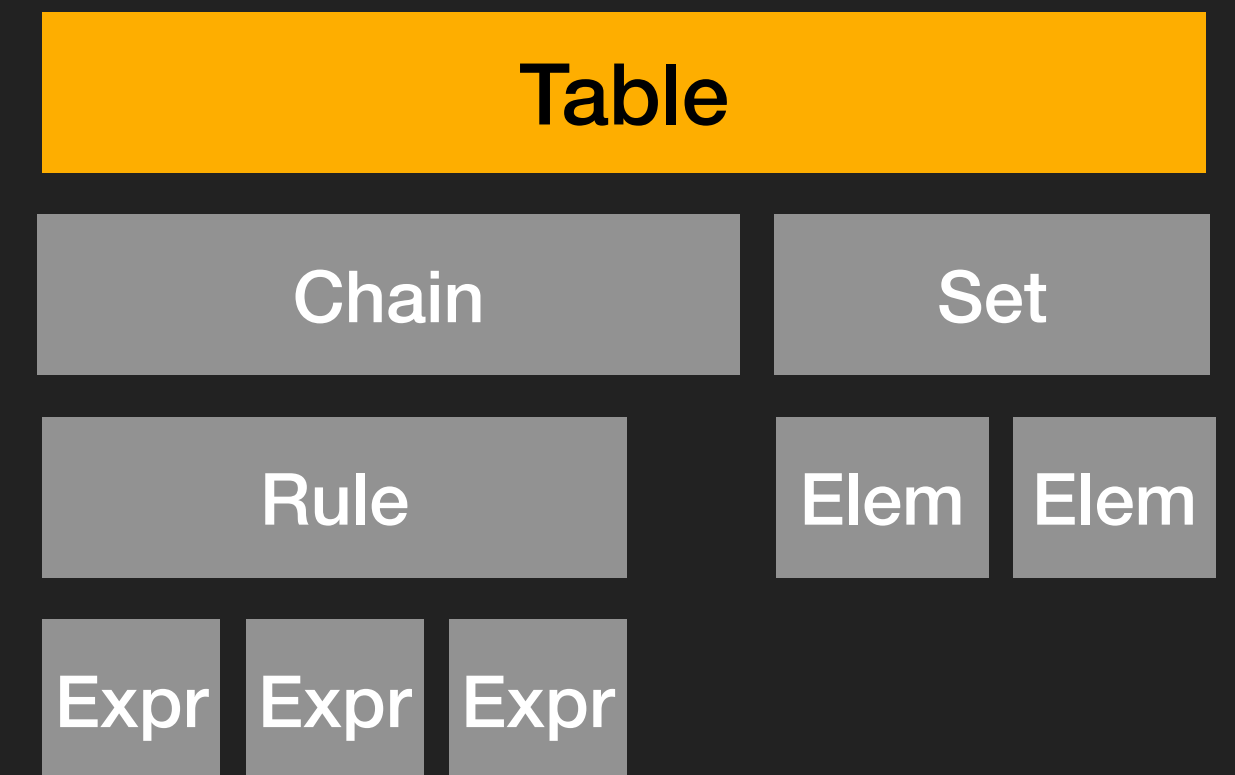

nftables core

- Core components
 - Table
 - Chain
 - Rule
 - Expression
 - Set



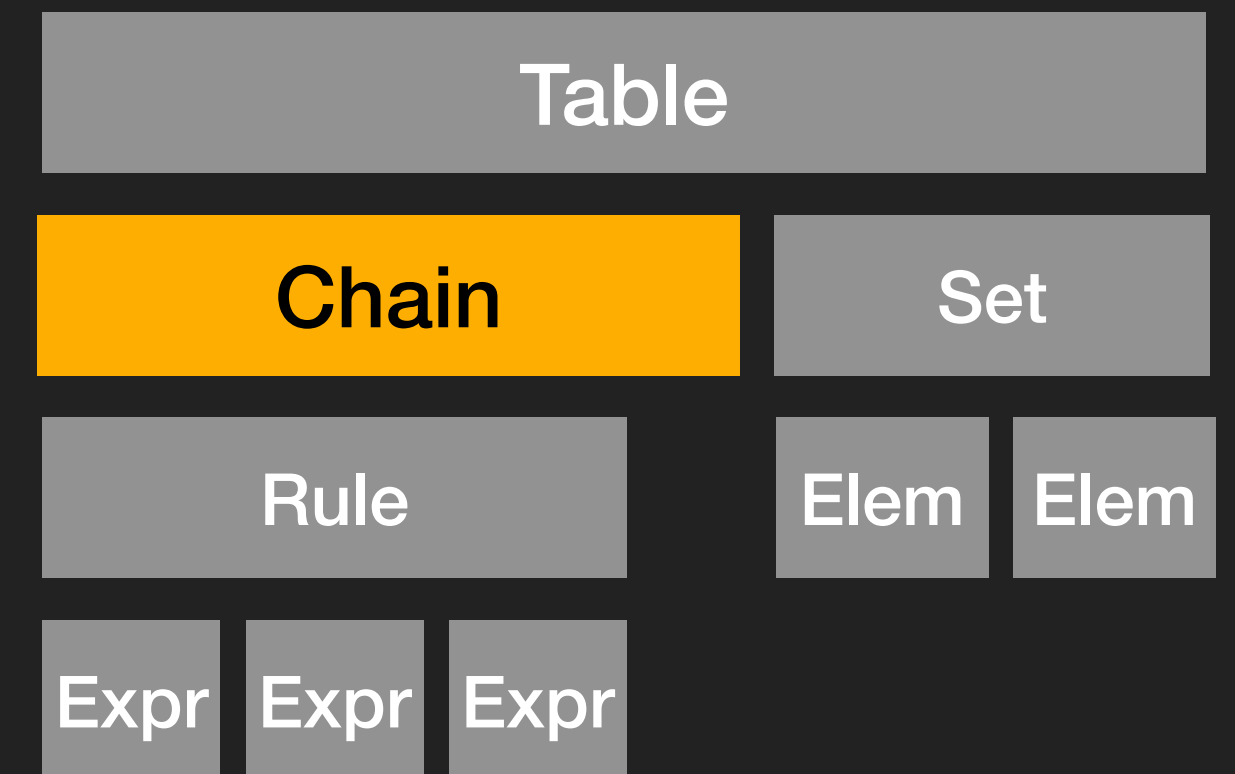
Table

- Top-level container for
 - Chain
 - Set
- Belong to a specific network family



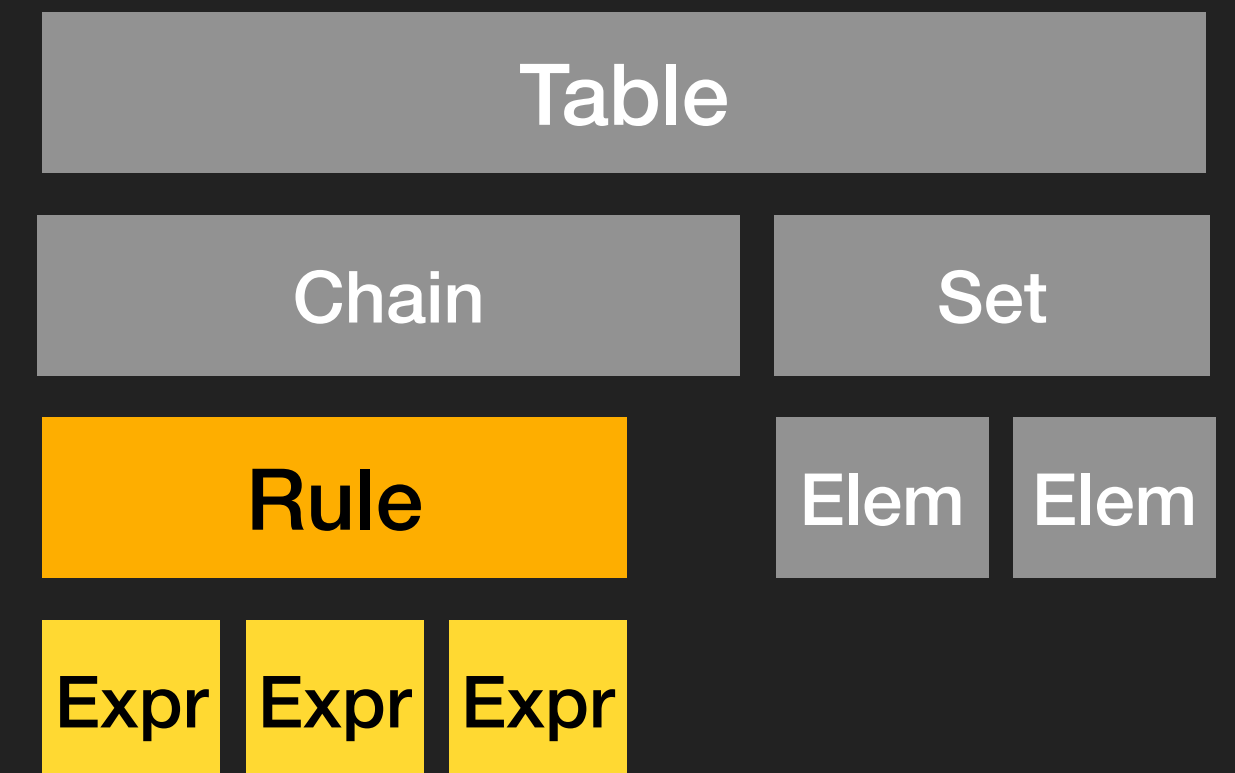
Chain

- Hook in–kernel packet filtering point
- Manage Rules
- Execute Expressions in Rules



Rule / Expression

- Rule consists of multiple Expression
 - act as a small function
- Expression is nftables VM instruction
 - manipulate VM registers
 - read/write data to packets



Set

- Group of some type
 - ip address
 - port
 - binary data
- Use as a map, mapping type A $\rightarrow$ type B
- Expressions can query data in runtime



The first bug

Different check

```
/* You cannot delete the same rule twice */  
if (nft_is_active_next(ctx->net, rule)) {  
    nft_deactivate_next(ctx->net, rule);  
    nft_use_dec(&ctx->chain->use);  
    return 0;  
}
```

nf_tables_delrule_deactivate

```
list_for_each_entry_safe(table, nt, &nft_net->tables) {  
    if (!nft_is_active_next(ctx->net, table))  
        continue;  
    // ...  
    err = nft_flush_table(ctx);  
}
```

nft_flush

```
list_for_each_entry(catchall, &set->catchall_list, list) {  
    ext = nft_set_elem_ext(set, catchall->elem);  
    if (!nft_is_active(net, ext))  
        continue;  
  
    kfree(elem->priv);  
    elem->priv = catchall->elem;  
    nft_set_elem_change_active(net, set, ext);  
    return 0;  
}
```

nft_setelem_catchall_deactivate

is_active

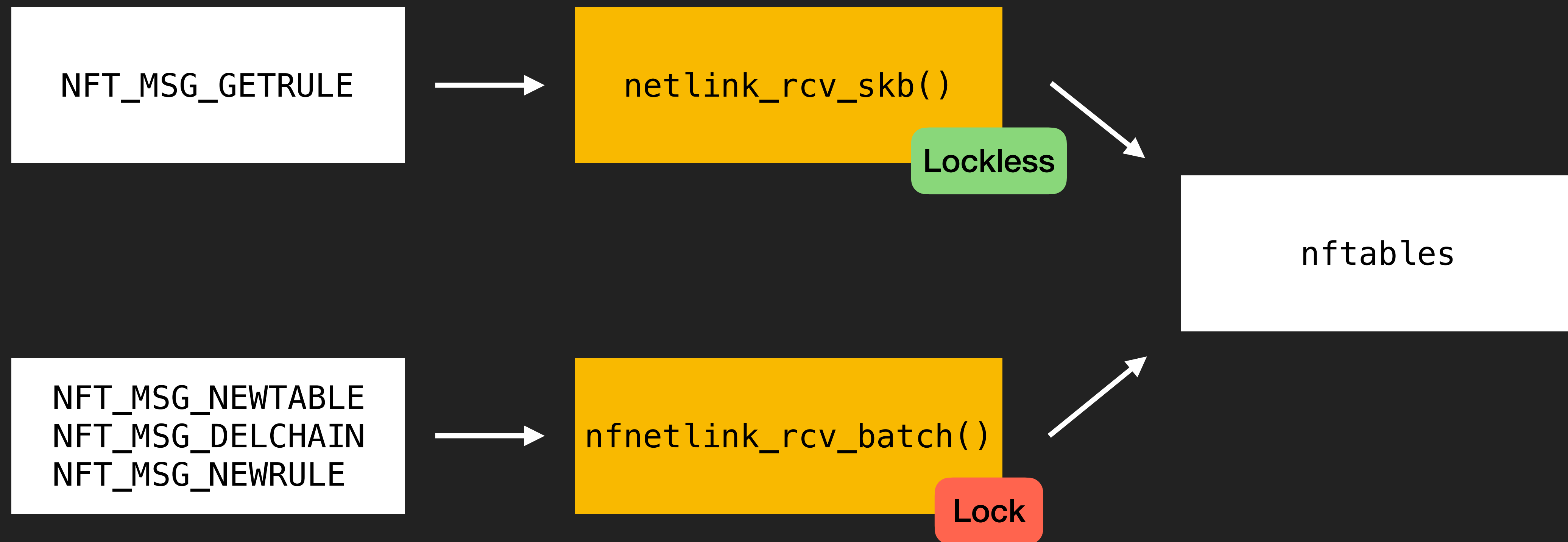
v.s.

is_active_next

Batch request

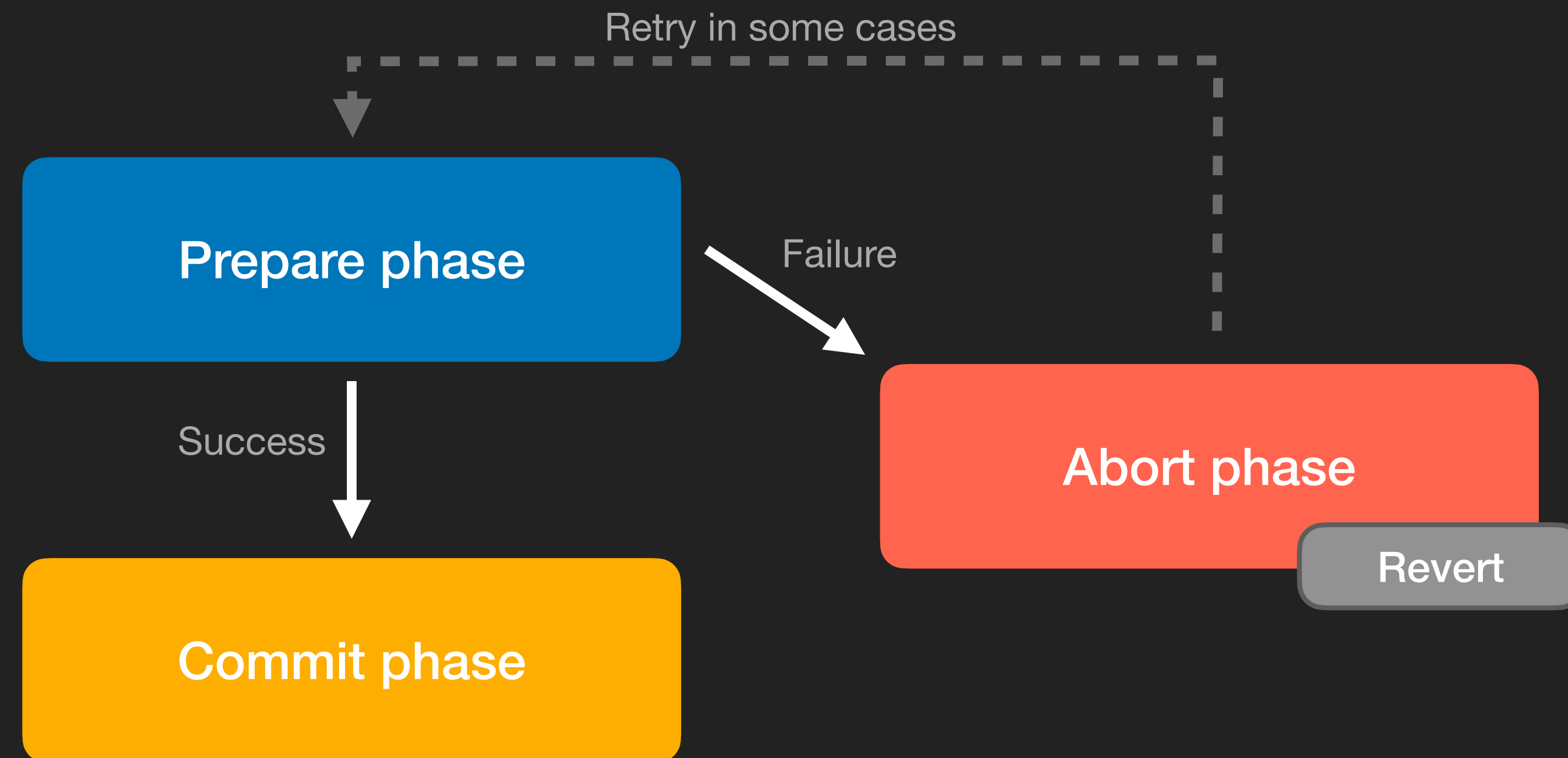


Batch request



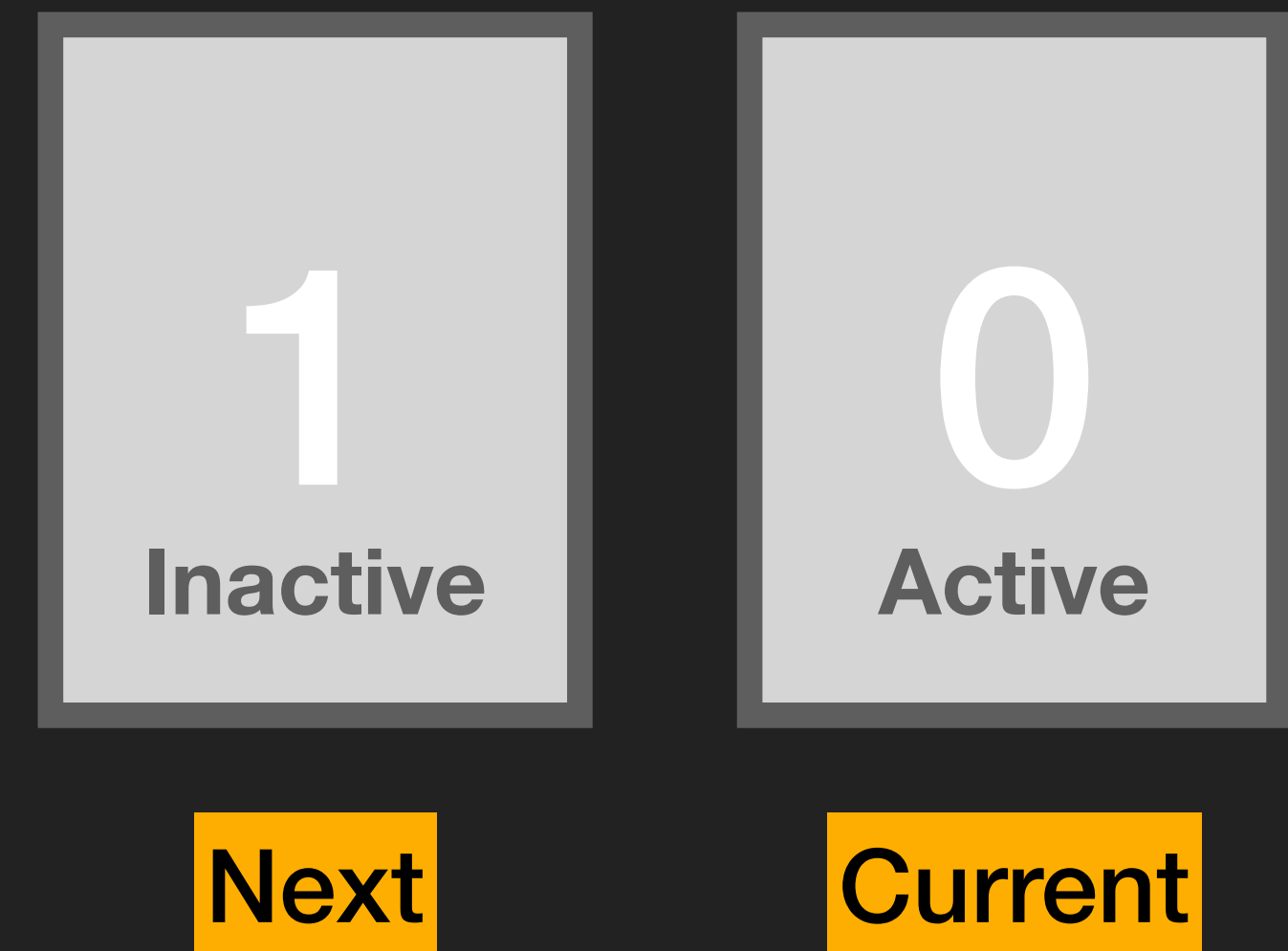
Batch request

- Batch request procedure
 - Prepare phase
 - Commit phase
 - Abort phase

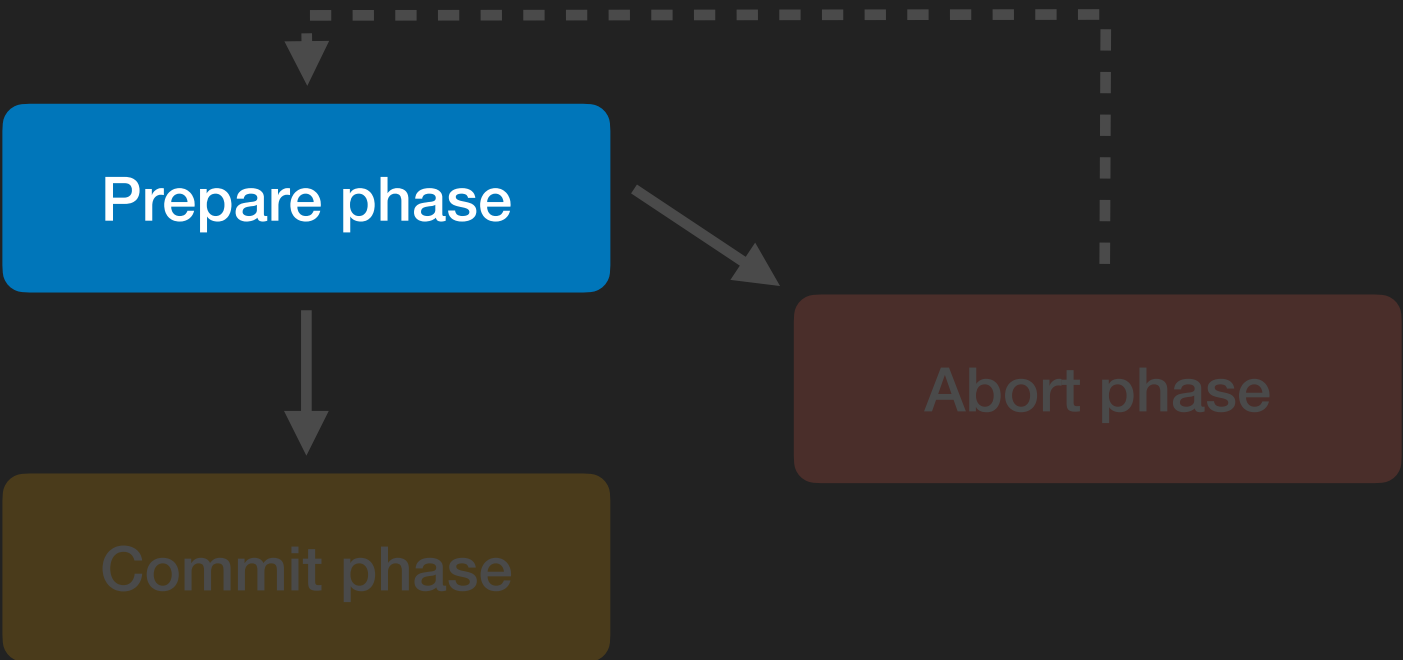
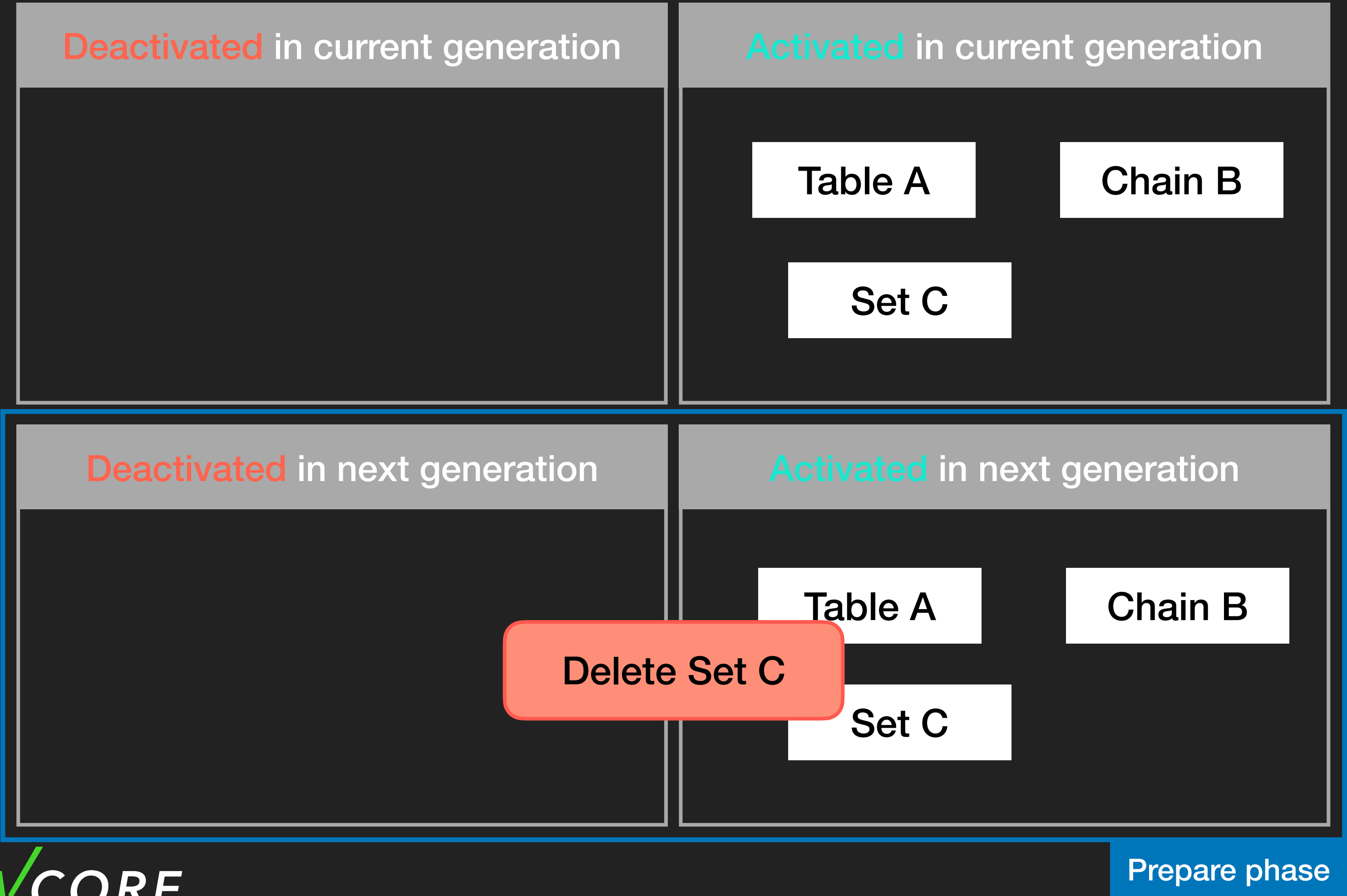


Lifetime management

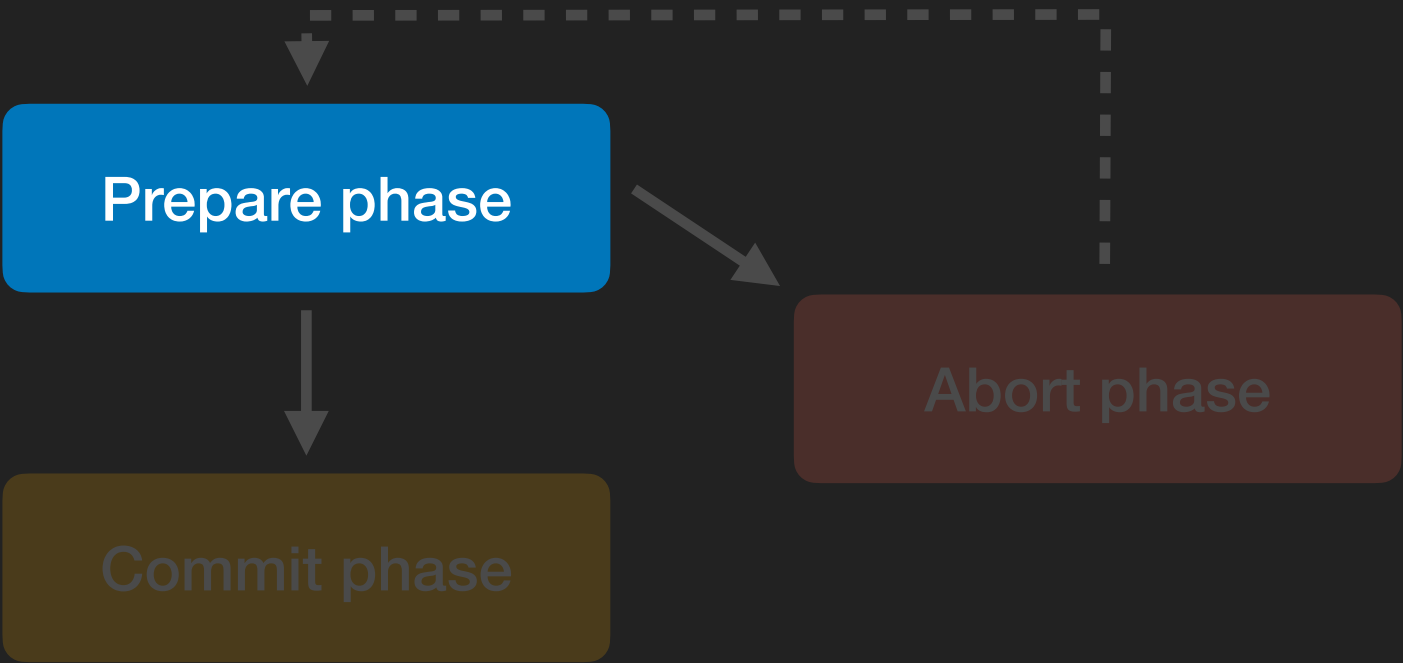
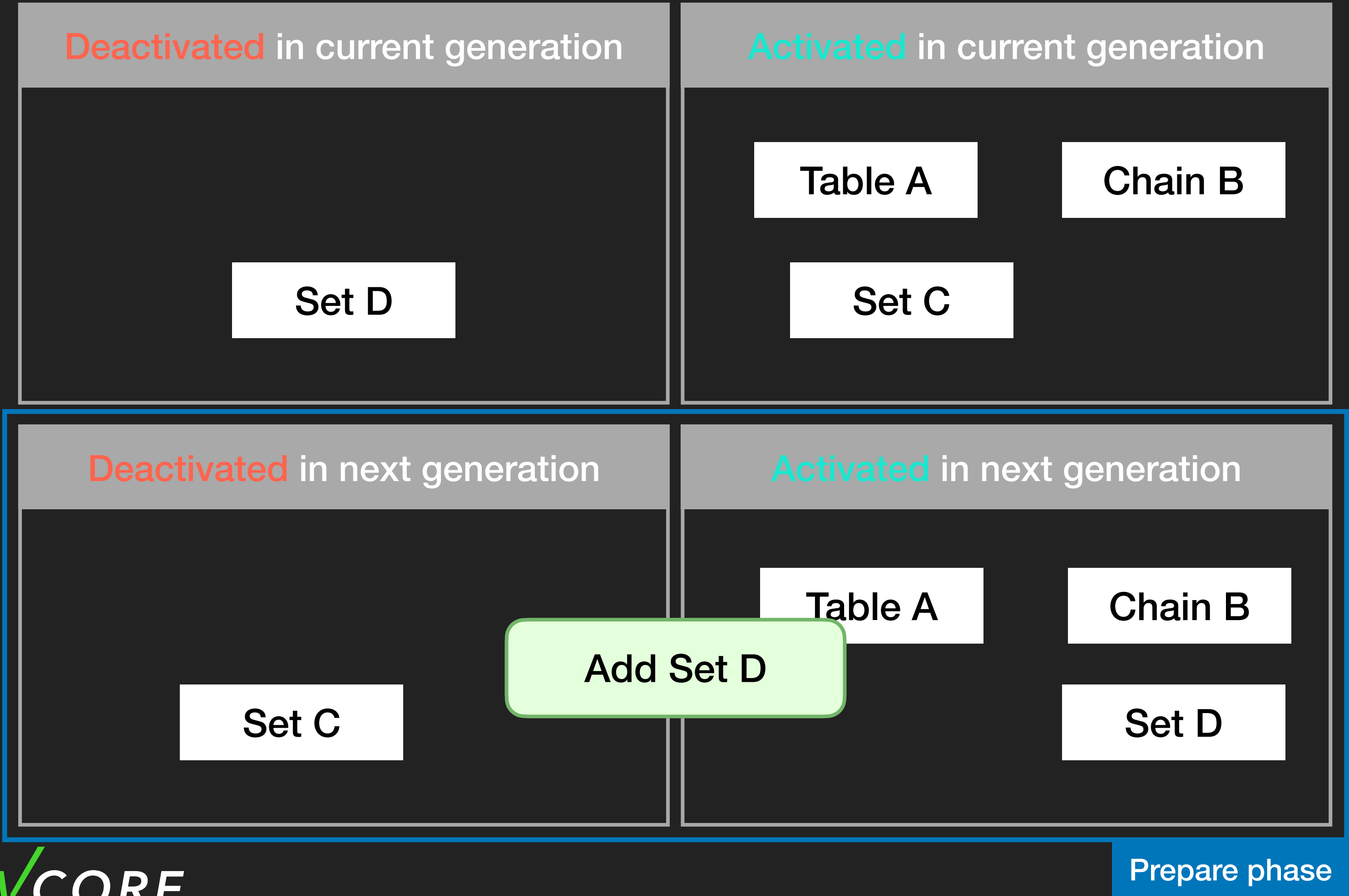
- An nftables object has two state variable
 - **Next generation**
 - new / delete / edit operation
 - **Current generation**
 - runtime / query operation
- Commit phase: **Next** → **Current**



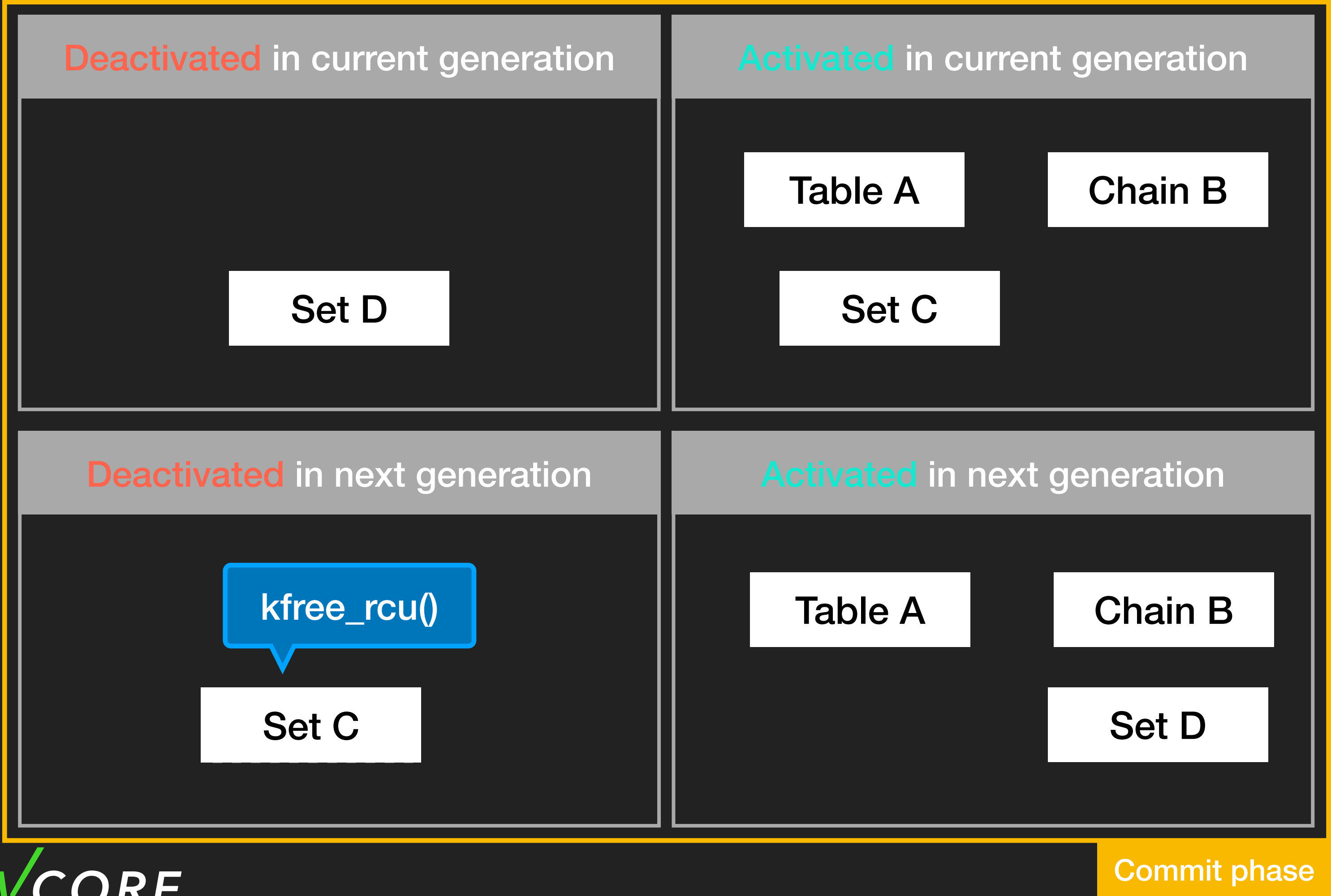
Batch request



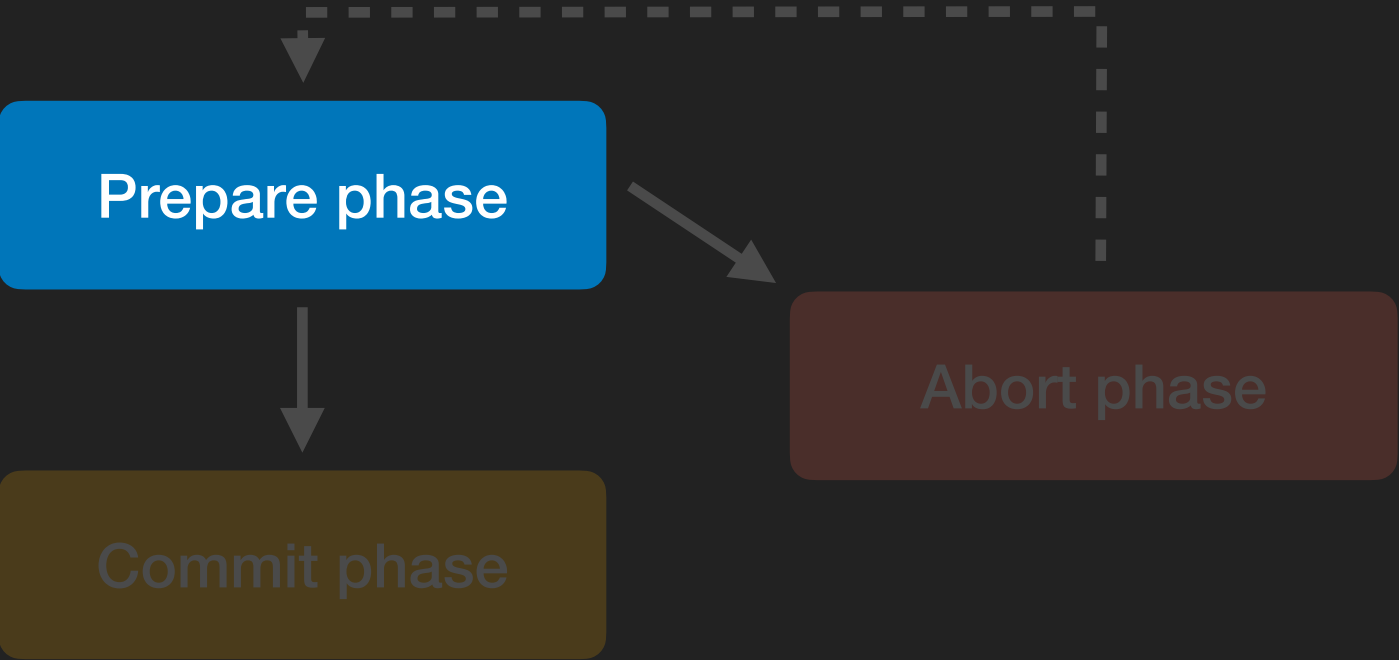
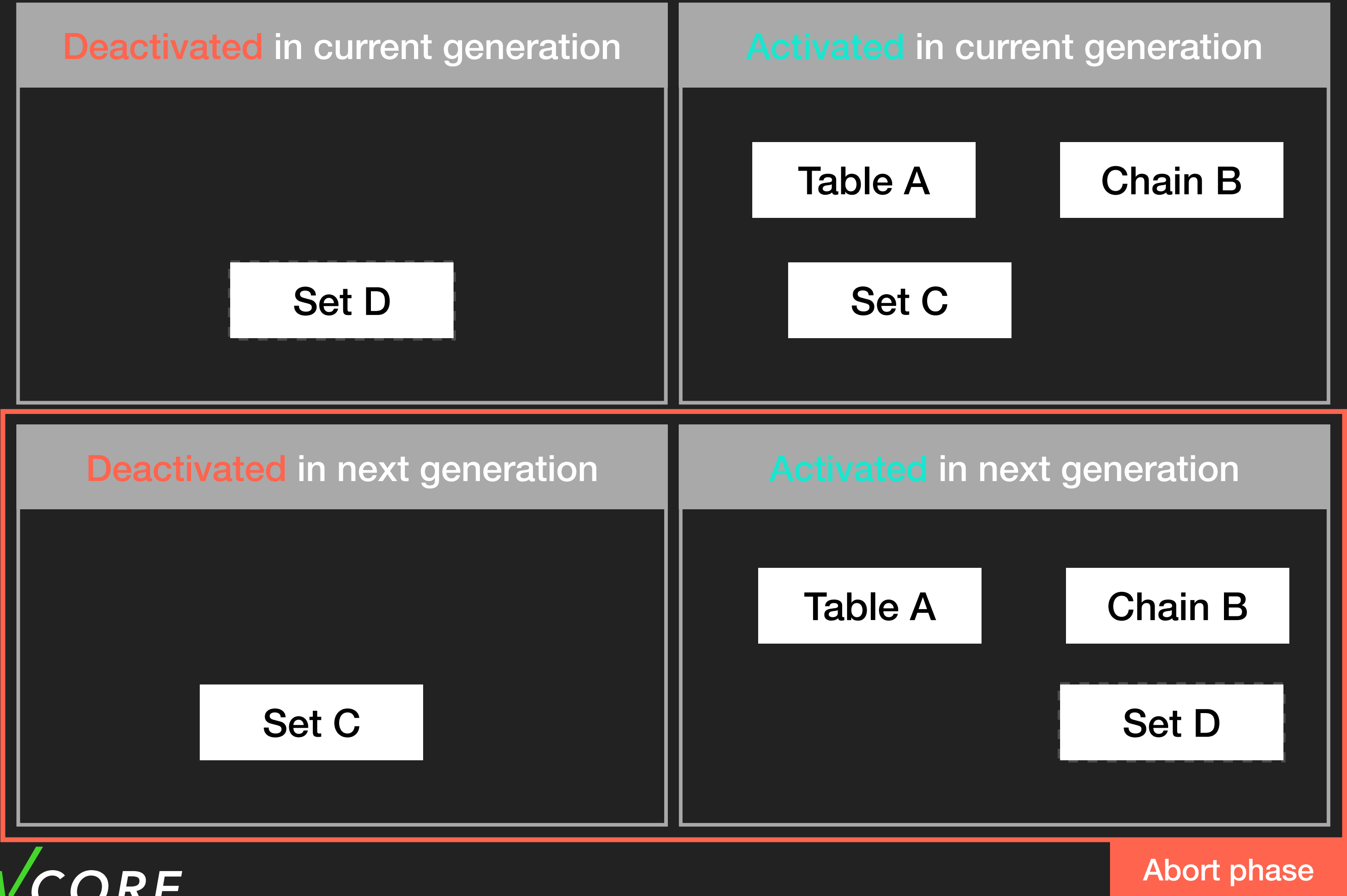
Batch request



Batch request



Batch request



Different check

```
/* You cannot delete the same rule twice */
if (nft_is_active_next(ctx->net, rule)) {
    nft_deactivate_next(ctx->net, rule);
    nft_use_dec(&ctx->chain->use);
    return 0;
}
```

nf_tables_delrule_deactivate

```
list_for_each_entry_safe(table, nt, &nft_net->tables) {
    if (!nft_is_active_next(ctx->net, table))
        continue;
    // ...
    err = nft_flush_table(ctx);
}
```

nft_flush

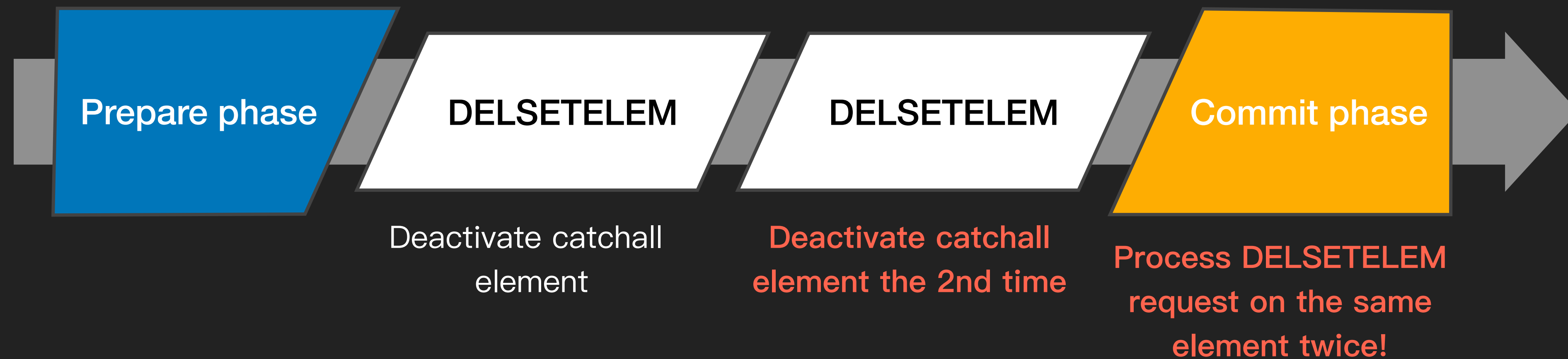
Check will always pass

```
list_for_each_entry(catchall, &set->catchall_list, list) {
    ext = nft_set_elem_ext(set, catchall->elem);
    if (!nft_is_active(net, ext))
        continue;

    kfree(elem->priv);
    elem->priv = catchall->elem;
    nft_set_elem_change_active(net, set, ext);
    return 0;
}
```

nft_setelem_catchall_deactivate

Exploit



Exploit



Set catchall element double free

Found my first bug 🎉

The next day

- Maintainer post patch to mailing list (???)

```
From: Pablo Neira Ayuso <pablo@netfilter.org>
To: netfilter-devel@vger.kernel.org
Subject: [PATCH nf] netfilter: nf_tables: check if catch-all set element is active in next generation
Date: Fri, 12 Jan 2024 23:31:56 +0100 [thread overview]
Message-ID: <20240112223156.311637-1-pablo@netfilter.org> (raw)
```

When deactivating the catch-all set element, check the state in the next generation that represents this transaction.

```
Fixes: aaa31047a6d2 ("netfilter: nftables: add catch-all set element support")
Signed-off-by: Pablo Neira Ayuso <pablo@netfilter.org>
```

```
net/netfilter/nf_tables_api.c | 2 +-
1 file changed, 1 insertion(+), 1 deletion(-)
```

```
diff --git a/net/netfilter/nf_tables_api.c b/net/netfilter/nf_tables_api.c
index 5e852a830a2b..8d4c0cf710be 100644
```

```
--- a/net/netfilter/nf_tables_api.c
```

```
+++ b/net/netfilter/nf_tables_api.c
```

```
@@ -6495,7 +6495,7 @@ static int nft_setelem_catchall_deactivate(const struct net *net,
```

```
    list_for_each_entry(catchall, &set->catchall_list, list) {
        ext = nft_set_elem_ext(set, catchall->elem);
-       if (!nft_is_active(net, ext))
+       if (!nft_is_active_next(net, ext))
            continue;

        kfree(elem->priv);
```

0-day → 1-day



-\$20k



Double free to LPE

- kernelCTF require bounty hunter to upload exploit script
- Reuse CVE-2023-4004 script to exploit double free primitive
 - Overlapping `nft_table.udata` and `nft_object` to exploit
 - Leak address / control RIP
 - Kudos to Ionial con

2 hours before
competition starts

The patch (again)

netfilter: nf_tables: check if catch-all set element is active in next generation

When deactivating the catch-all set element, check the state in the next generation that represents this transaction.

This bug uncovered after the recent removal of the element busy mark a2dd0233cbc4 ("netfilter: nf_tables: remove busy mark and gc batch API").

Fixes: aaa31047a6d2 ("netfilter: nftables: add catch-all set element support")

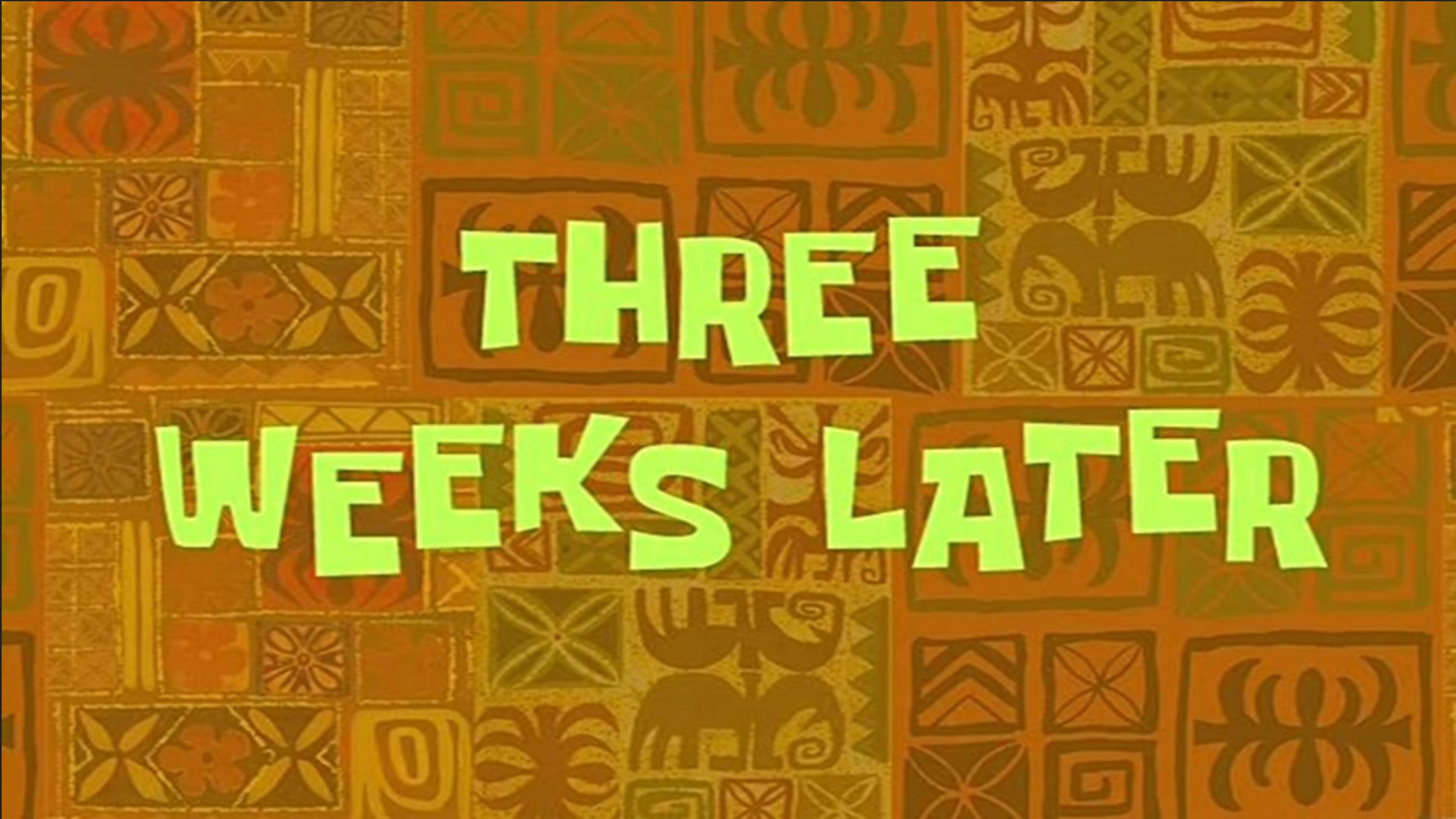
Cc: stable@vger.kernel.org

Reported-by: lonial con <kongln9170@gmail.com>

Signed-off-by: Pablo Neira Ayuso <pablo@netfilter.org>

Bug collision





**THREE
WEEKS LATER**

The second bug

Interesting commit

- netfilter: nf_tables: validate family when identifying table via handle

```
static struct nft_table *nft_table_lookup_byhandle(const struct net *net,  
                                                    const struct nlattr *nla,  
                                                    u8 genmask, u32 nlpid)  
+ int family, u8 genmask, u32 nlpid)  
{  
    struct nftables_pernet *nft_net;  
    struct nft_table *table;  
  
    nft_net = nft_pernet(net);  
    list_for_each_entry(table, &nft_net->tables, list) {  
        if (be64_to_cpu(nla_get_be64(nla)) == table->handle &&  
+         table->family == family &&  
            nft_active_genmask(table, genmask)) {  
                if (nft_table_has_owner(table) &&  
                    nlpid && table->nlpid != nlpid)
```

```
static int nf_tables_deltable(struct sk_buff *skb, const struct nfnl_info *info,
                             const struct nlattr * const nla[])
{
    u8 family = info->nfmsg->nfgen_family;
    struct nft_ctx ctx;

    // ...
    if (nla[NFTA_TABLE_HANDLE]) {
        attr = nla[NFTA_TABLE_HANDLE];
        table = nft_table_lookup_byhandle(net, attr, genmask,
                                           NETLINK_CB(skb).portid);
    }

    // ...

    ctx.family = family;
    ctx.table = table;
    return nft_flush_table(&ctx);
}
```

```

static int nf_tables_deltable
{
    const struct nft_info *info;
    u8 family = info->nfmsg->family;
    struct nft_ctx ctx;

    // ...
    if (nla[NFTA_TABLE_HANDLE] != NULL) {
        attr = nla[NFTA_TABLE_HANDLE];
        table = nft_table_lookup(ctx.family, attr);
    }

    // ...

    ctx.family = family;
    ctx.table = table;

    return nft_flush_table(&ctx);
}

```

```

static int nft_flush_table(struct nft_ctx *ctx)
{
    list_for_each_entry(chain, &ctx->table->chains, list) {
        err = nft_delrule_by_chain(ctx);
    }

    list_for_each_entry_safe(set, ns, &ctx->table->sets, list) {
        err = nft_delset(ctx, set);
    }

    list_for_each_entry_safe(flowtable, nft, &ctx->table->flowtables, list) {
        err = nft_delflowtable(ctx, flowtable);
    }

    list_for_each_entry_safe(obj, ne, &ctx->table->objects, list) {
        err = nft_delobj(ctx, obj);
    }

    list_for_each_entry_safe(chain, nc, &ctx->table->chains, list) {
        err = nft_delchain(ctx);
    }

    err = nft_deltable(ctx);
out:
    return err;
}

```


It's grep time!

`ctx->family`

✓ SEARCH

> ctx->family Aa _ab_ *

50 results in 16 files - [Open in editor](#)

> C nft_compat.c net/netfilter	9
> C nft_connlimit.c net/netfilter	2
> C nft_ct.c net/netfilter	16
> C nft_exthdr.c net/netfilter	1
> C nft_flow_offload.c net/netfilter	2
> C nft_log.c net/netfilter	2
> C nft_masq.c net/netfilter	1
> C nft_meta.c net/netfilter	4
> C nft_nat.c net/netfilter	2
> C nft_osf.c net/netfilter	1
> C nft_queue.c net/netfilter	1
> C nft_redir.c net/netfilter	1
> C nft_socket.c net/netfilter	1
> C nft_synproxy.c net/netfilter	5
> C nft_tproxy.c net/netfilter	1
> C nft_xfrm.c net/netfilter	1

Trigger path

```
static void nft_log_destroy(const struct nft_ctx *ctx,
                           const struct nft_expr *expr)
{
    struct nft_log *priv = nft_expr_priv(expr);
    struct nf_loginfo *li = &priv->loginfo;

    if (priv->prefix != nft_log_null_prefix)
        kfree(priv->prefix);

    if (li->u.log.level == NFT_LOGLEVEL_AUDIT)
        return;

    nf_logger_put(ctx->family, li->type);
}
```

```
// NFPROTO_NUMPROTO = 13
// NF_LOG_TYPE_MAX = 2
static struct nf_logger __rcu *loggers[NFPROTO_NUMPROTO][NF_LOG_TYPE_MAX];
```

Array length = 13

```
void nf_logger_put(int pf, enum nf_log_type type)
{
    struct nf_logger *logger;

    if (pf == NFPROTO_INET) {
        nf_logger_put(NFPROTO_IPV4, type);
        nf_logger_put(NFPROTO_IPV6, type);
        return;
    }

    BUG_ON(loggers[pf][type] == NULL);

    rcu_read_lock();
    logger = rcu_dereference(loggers[pf][type]);
    module_put(logger->me);
    rcu_read_unlock();
}
```

Value range: 0 ~ 255

Trigger path

```
static void nft_log_destroy(const struct nft_ctx *ctx,
                           const struct nft_expr *expr)
{
    struct nft_log *priv = nft_expr_priv(expr);
    struct nf_loginfo *li = &priv->loginfo;

    if (priv->prefix != nft_log_null_prefix)
        kfree(priv->prefix);

    if (li->u.log.level == NFT_LOGLEVEL_AUDIT)
        return;

    nf_logger_put(ctx->family, li->type);
}
```

```
// NFPROTO_NUMPROTO = 13
// NF_LOG_TYPE_MAX = 2
static struct nf_logger
```

```
...[TYPE_MAX];
```

Global array Out-Of-Bound access

```
..._type type)
{
    if (ctx->family == AF_INET) {
        nf_logger_put(NFPROTO_IPV4, type);
        nf_logger_put(NFPROTO_IPV6, type);
    }
    return;
}
```

```
BUG_ON(loggers[pf][type] == NULL);

rcu_read_lock();
logger = rcu_dereference(loggers[pf][type]);
module_put(logger->me);
rcu_read_unlock();
}
```

Trigger path

```
static void nft_log_destroy(const struct nft_ctx *ctx,
                           const struct nft_expr *expr)
{
    struct nft_log *priv = nft_expr_priv(expr);
    struct nf_loginfo *li = &priv->loginfo;

    if (priv->prefix != nft_log_null_prefix)
        kfree(priv->prefix);

    if (li->u.log.level == NFT_LOGLEVEL_AUDIT)
        return;

    nf_logger_put(ctx->family, li->type);
}
```

```
// NFPROTO_NUMPROTO = 13
// NF_LOG_TYPE_MAX = 2
static struct nf_logger __rcu *loggers[NFPROTO_NUMPROTO][NF_LOG_TYPE_MAX];
```

```
void nf_logger_put(int pf, enum nf_log_type type)
{
    struct nf_logger *logger;

    if (pf == NFPROTO_INET) {
        nf_logger_put(NFPROTO_IPV4, type);
        nf_logger_put(NFPROTO_IPV6, type);
        return;
    }

    BUG_ON(loggers[pf][type] == NULL);

    rcu_read_lock();
    logger = rcu_dereference(loggers[pf][type]);
    module_put(logger->me);
    rcu_read_unlock();
}
```

```
void module_put(struct module *module)
{
    int ret;

    if (module) {
        preempt_disable();
        ret = atomic_dec_if_positive(&module->refcnt);
        WARN_ON(ret < 0); /* Failed to put refcount */
        trace_module_put(module, _RET_IP_);
        preempt_enable();
    }
}
```

2nd Deref

1st Deref



Exploit primitives

- Maximum pf = ctx->family = 255
- Hard to use out-of-bounds reference count decrement
 - `loggers[pf][0 or 1]->me->refcnt--`

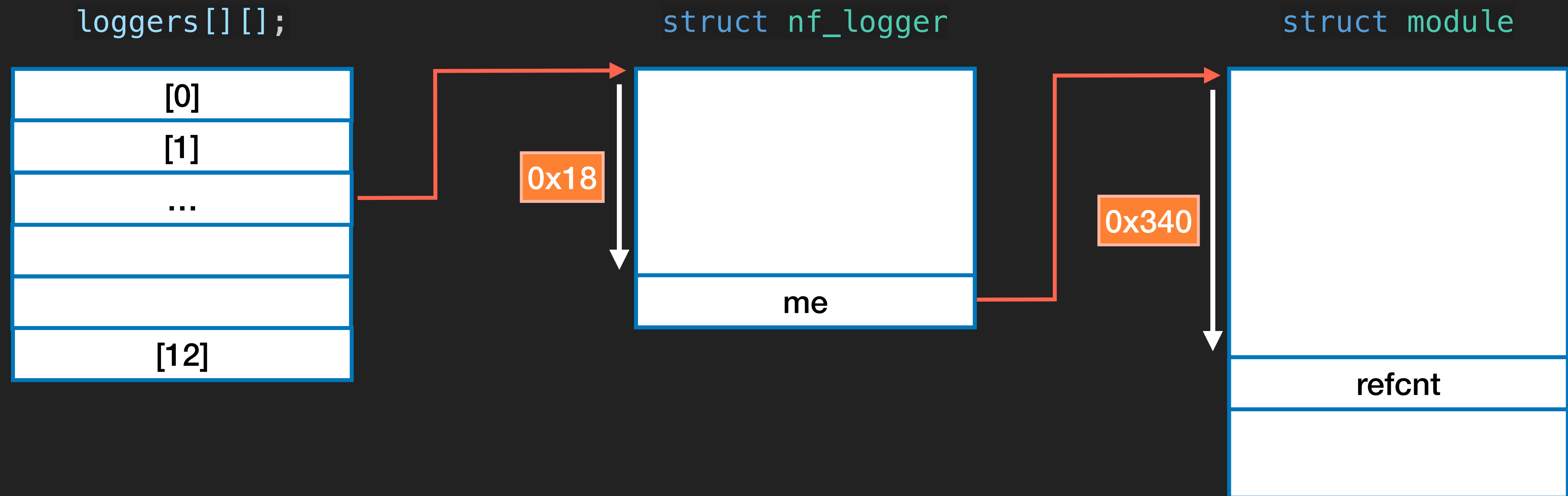
Global variables

- What do we have on the global?

```
gef> x/100gx loggers
0xffffffff83d84cc0 <loggers>: 0x0000000000000000
...
0xffffffff83d84d70 <sysctl_nf_log_all_netns>: 0x0000001600000000
0xffffffff83d84d78 <nfnl_acct_net_id>: 0x0000001800000017
0xffffffff83d84d80 <nfnl_log_net_id>: 0xffffffff830f52c1
0xffffffff83d84da0 <nfnl_log_net_id>: 0x0000000000000019
0xffffffff83d84da8 <size>: 0x0000000000000000
0xffffffff83d84db0 <nf_conntrack_max>: 0x00001e0000001e00
0xffffffff83d84db8 <nf_conntrack_locks_all>: 0x0000000000000000
0xffffffff83d84dc0 <nf_conntrack_cachep>: 0xffffffe8803bcf8c0
0xffffffff83d84dc8 <nf_conntrack_hash>: 0xffff88800a5a0000
0xffffffff83d84dd0 <nf_conntrack_htable_size_user>: 0x0000001a00001e00
0xffffffff83d84dd8 <enable_hooks>: 0x0000000000000000
0xffffffff83d84de0 <nf_ct_expect_cachep>: 0xffffffe8803bcf640
0xffffffff83d84de8 <nf_ct_expect_max>: 0x0000000000000078
0xffffffff83d84df0 <nf_ct_expect_hash>: 0xffffffe8802391000
0xffffffff83d84df8 <nf_ct_expect_hsize>: 0x0000000000000200
```

Normal access path

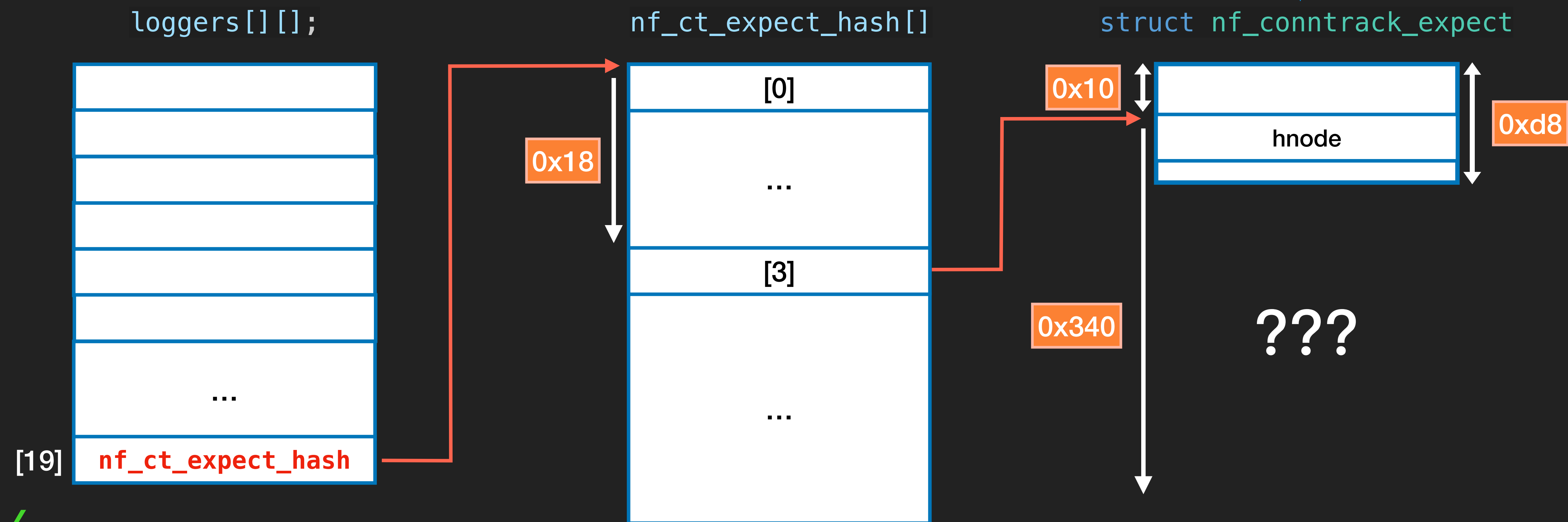
- `loggers[pf][0/1] -> me -> refcnt--`



Out-Of-Bounds access path

- `loggers[pf][0/1] -> me -> refcnt --`

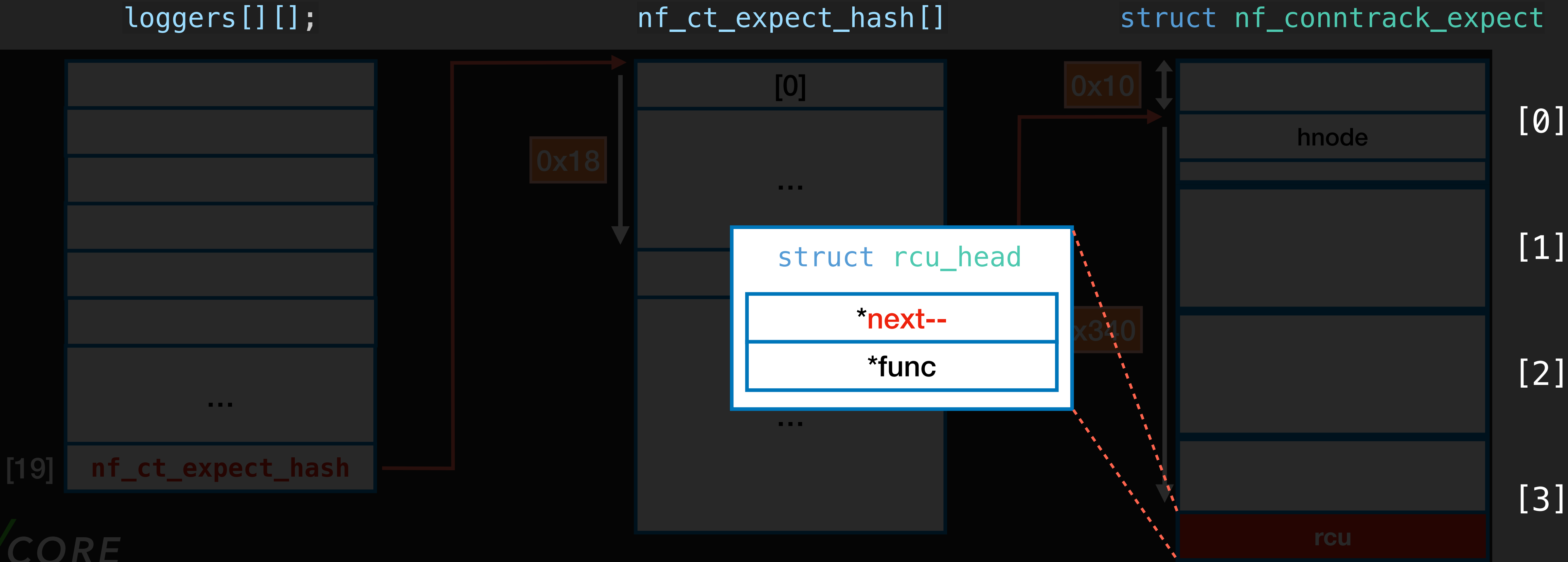
Allocate in dedicated kmem_cache



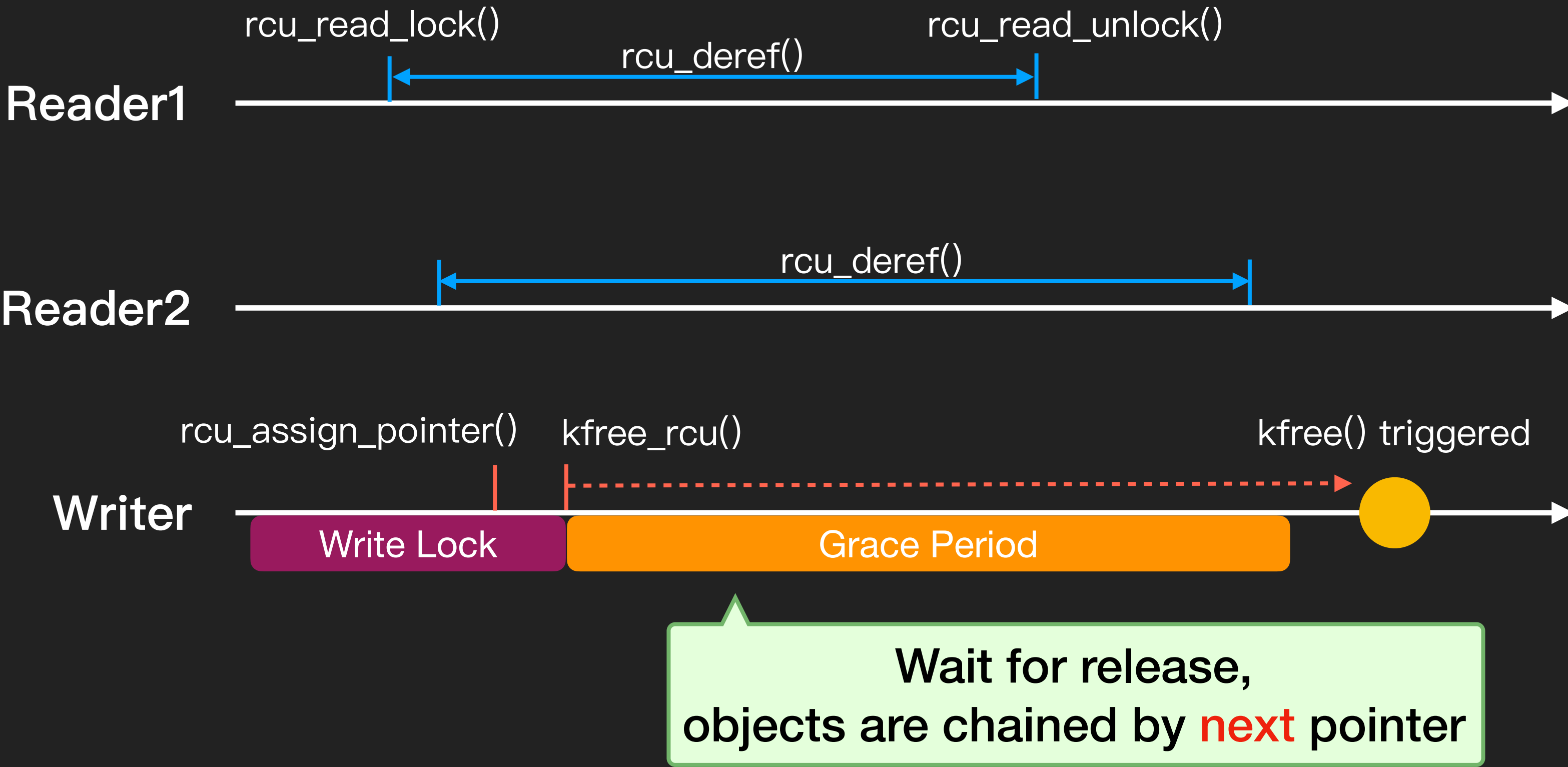
Out-Of-Bounds access path

- `loggers[pf][0/1] -> me -> refcnt--`

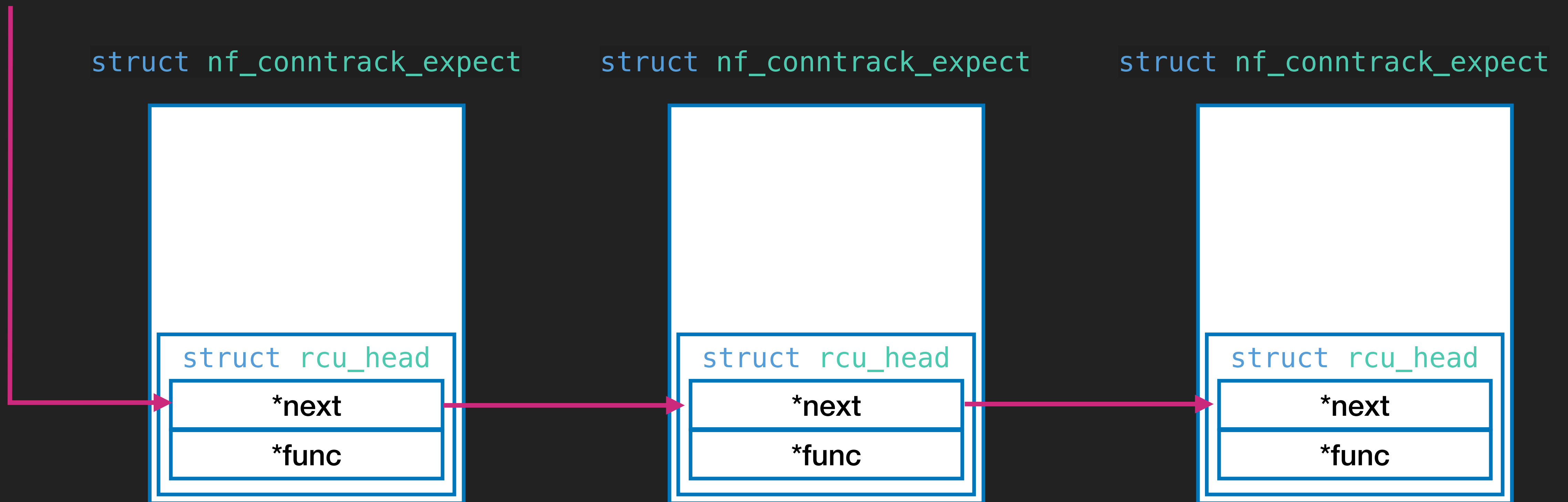
Allocate in dedicated kmem_cache



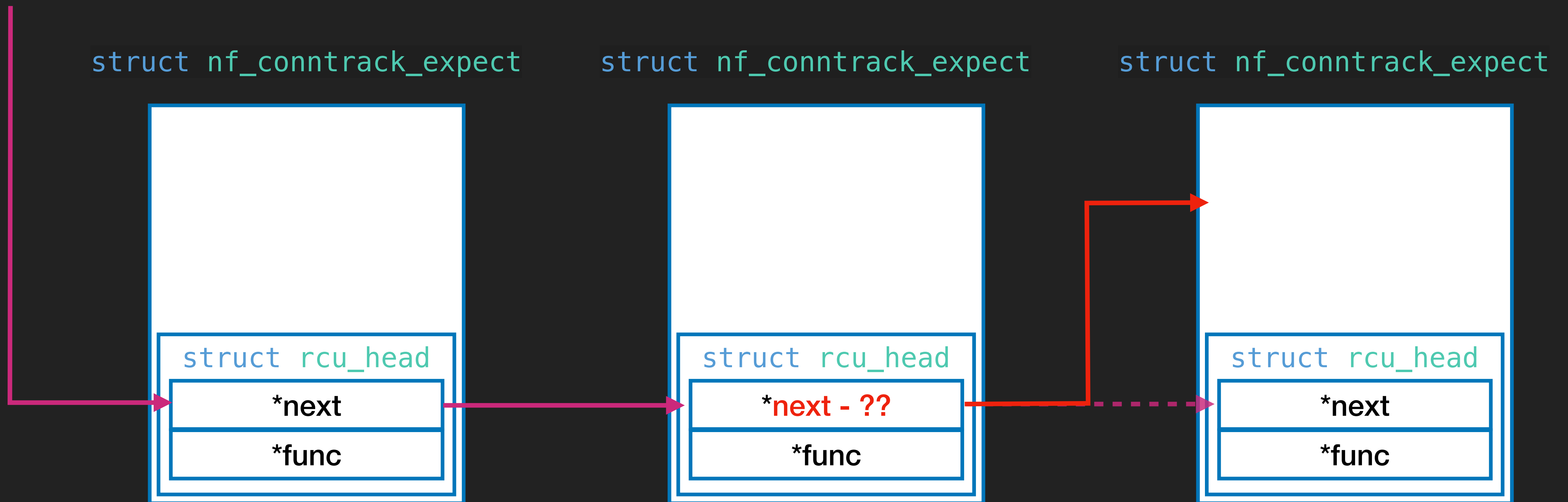
RCU 101



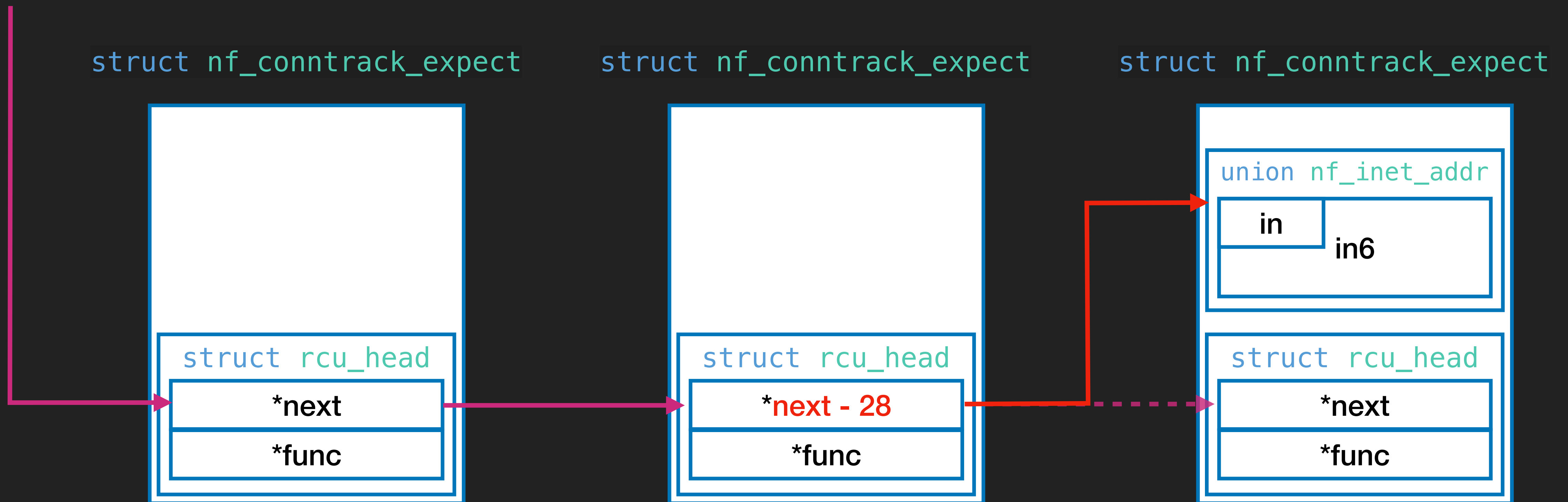
Exploit strategy



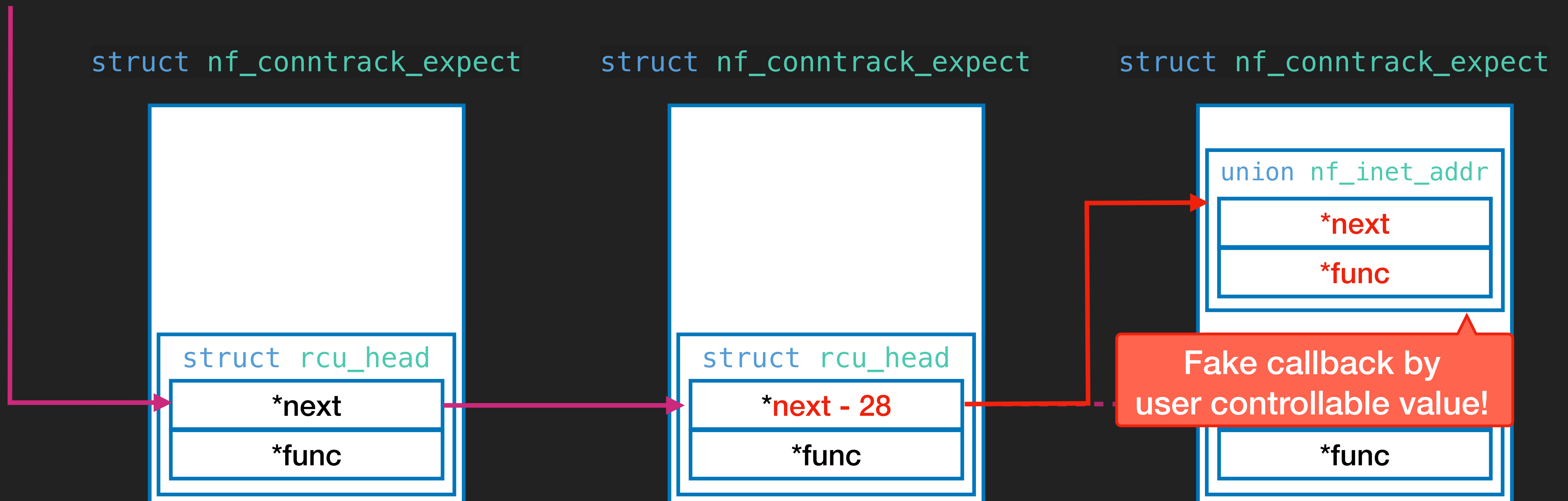
Exploit strategy



Exploit strategy



Exploit strategy



Exploit roadblocks

- `nf_ct_expect_hash[3]` exist
 - Unpredictable hash
 - The OOB written expect object need to be in grace period
 - Small time window
- ➔ Exploit by bruteforce + race

Exploit

```
if (p->max_expected &&  
    master_help->expecting[expect->class] >= p->max_expected) {  
    evict_oldest_expect(master, expect);  
}
```

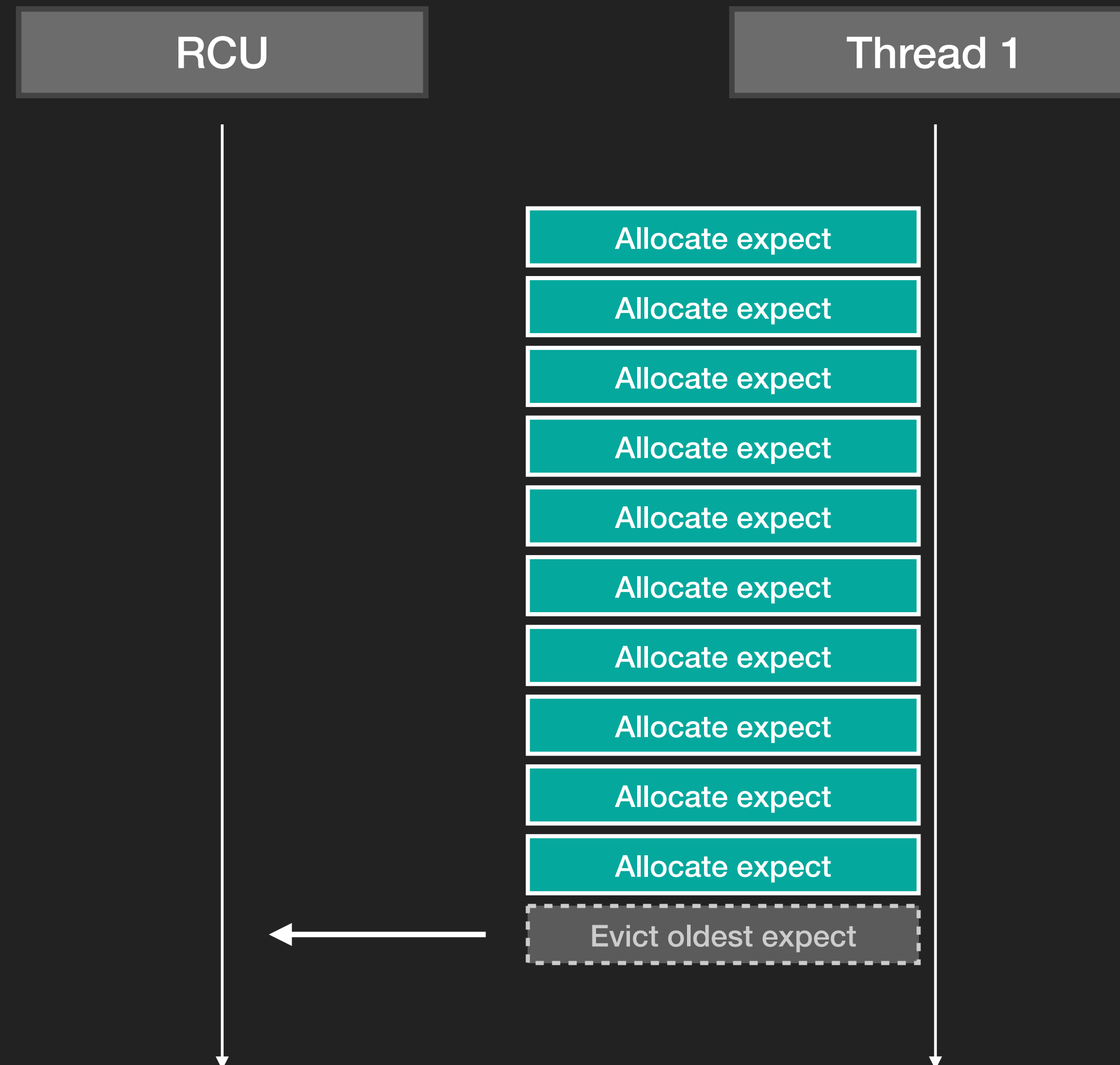
Exceeded max_expected,
evict oldest expectation

Thread 1

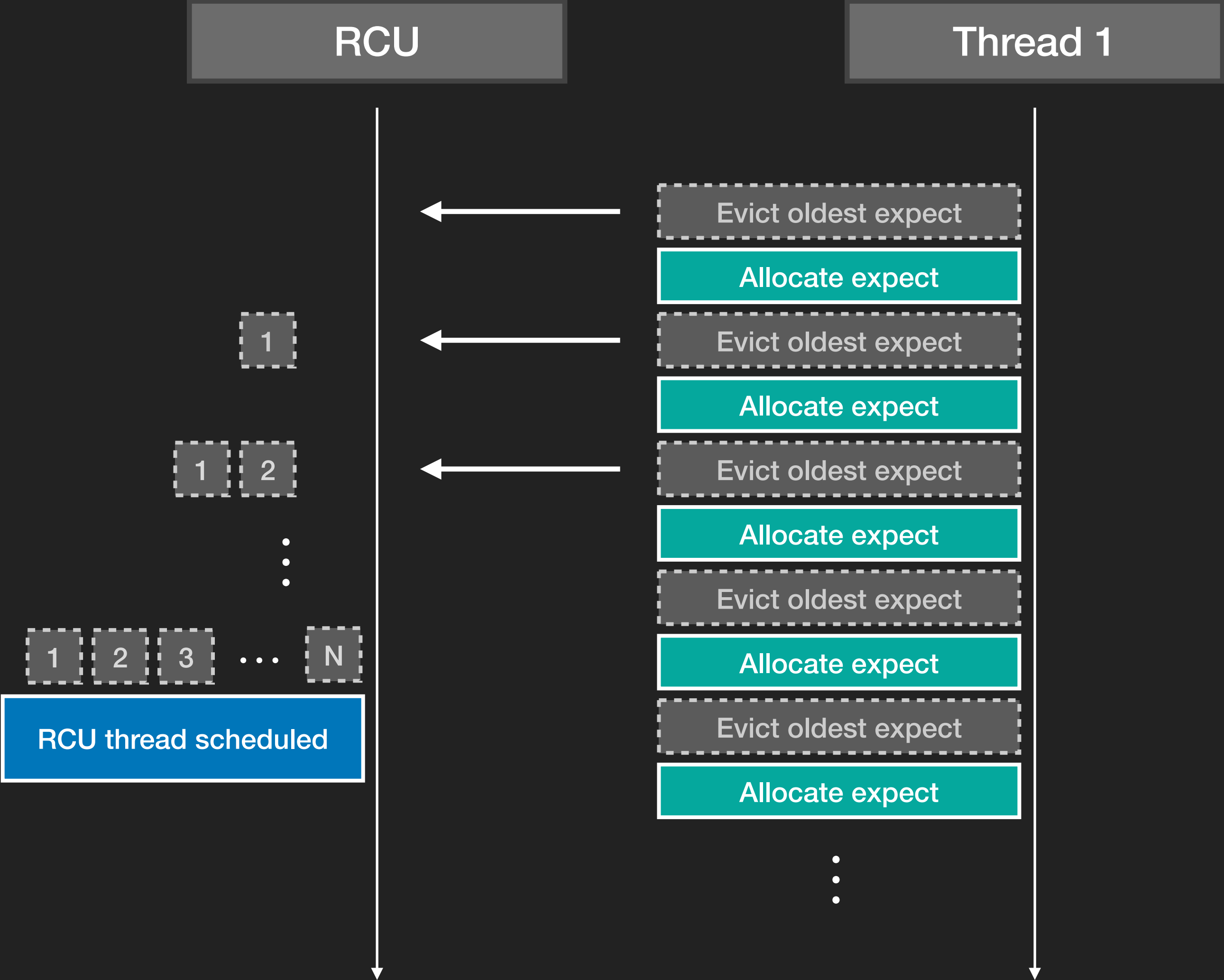


Tight loop creating
expect objects

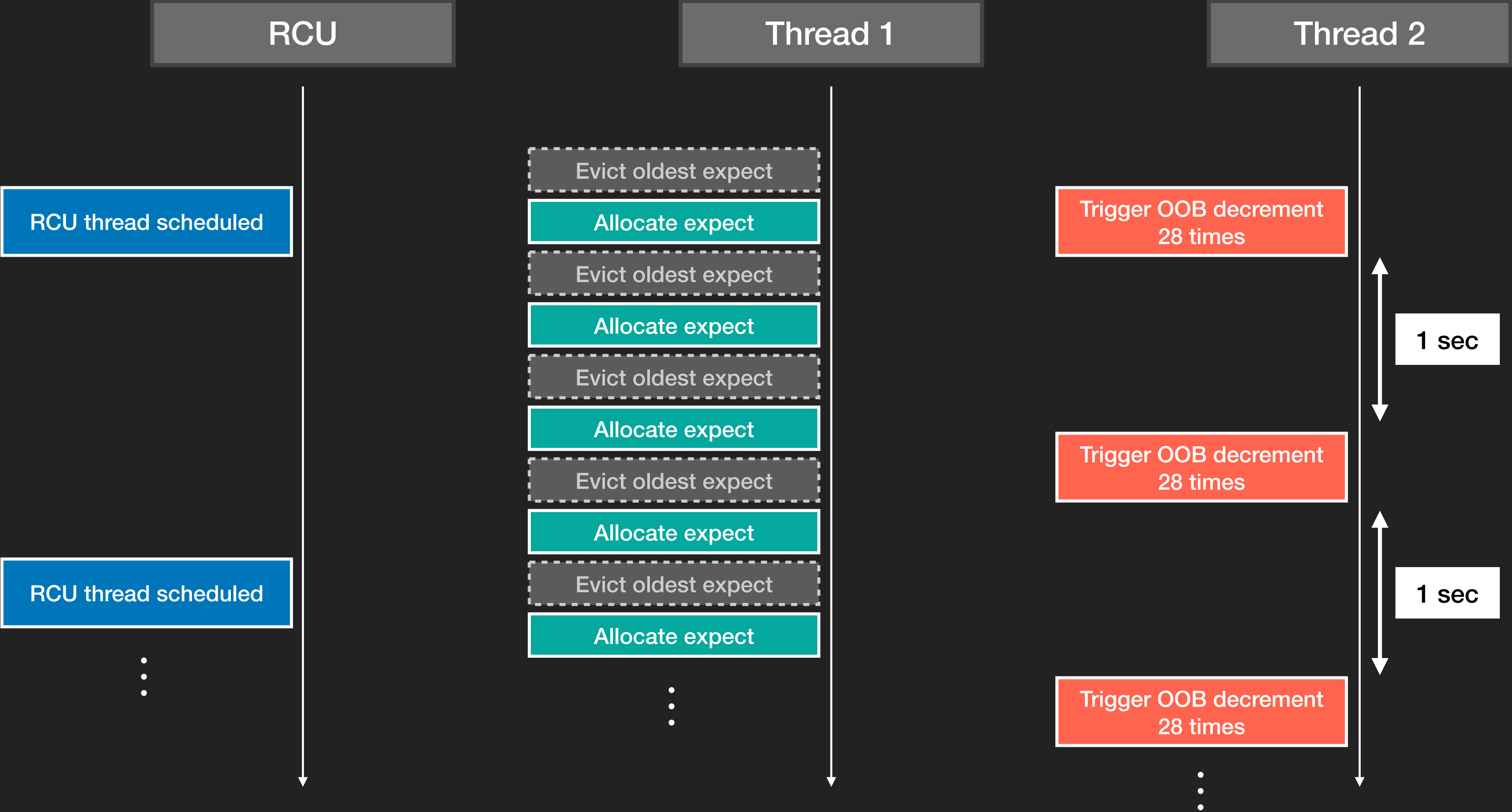
Exploit



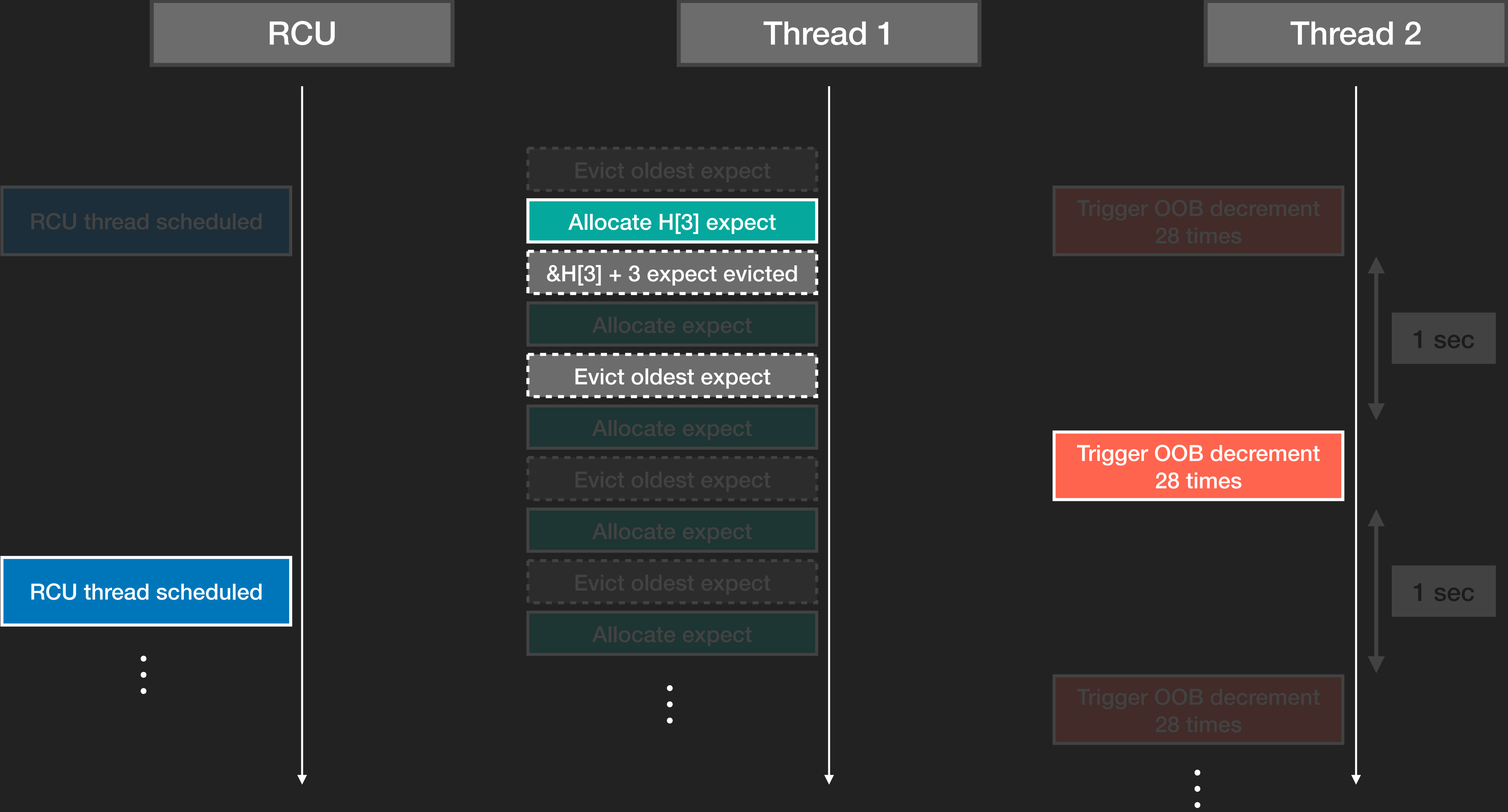
Exploit



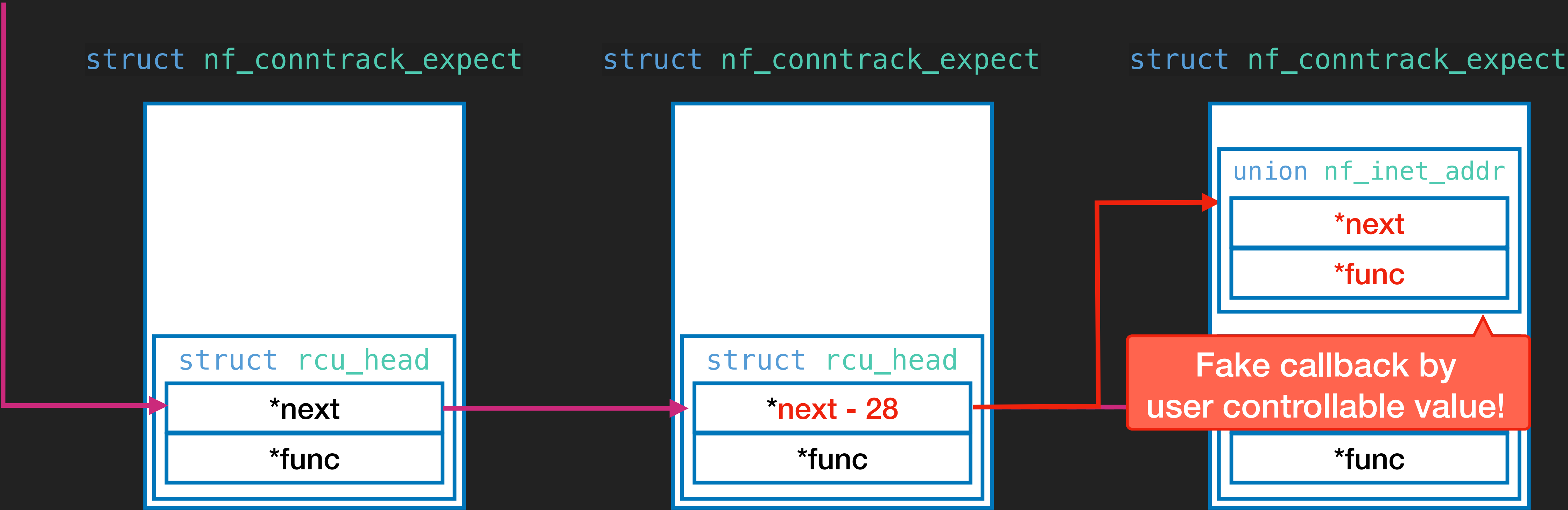
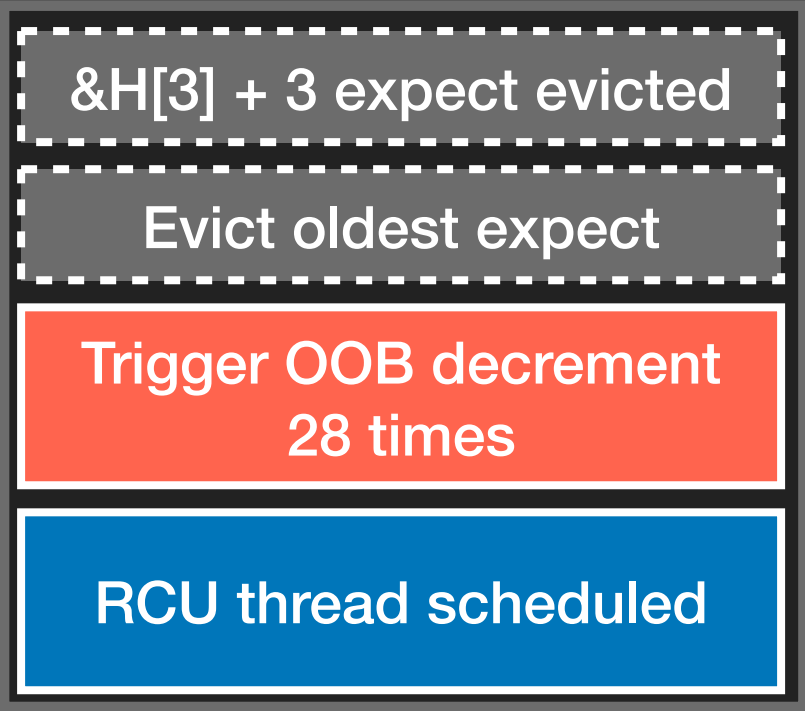
Exploit



Exploit



Race scenario



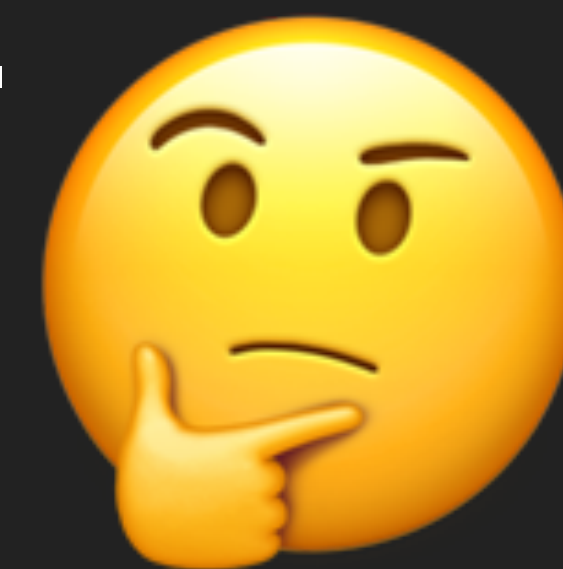
Try it on local build...


```
general protection fault, probably for non-canonical address 0x4141414141414141
CPU: 0 PID: 15 Comm: ksoftirqd/0 Not tainted 6.1.55+ #3
Hardware name: QEMU Standard PC (i440FX + PIIX, 1996), BIOS 1.16.2-debian-1.16.
RIP: 0010:rcu_core+0x2ca/0x7a0
Code: 89 c7 48 85 c0 0f 84 82 01 00 00 48 8b 47 08 49 89 dd 48 c7 47 08 00 00 0
RSP: 0018:ffffc900000083e20 EFLAGS: 00010207
RAX: 4141414141414141 RBX: 00000000000000005 RCX: 0000000008012000d
RDX: 4141414141414141 RSI: fffffffe8000210790 RDI: fffffffe8804b7f184
RBP: ffff88803dc326c0 R08: 0000000000000000 R09: 0000000008012000d
R10: fffffffe8804b7f948 R11: 00000000000000001 R12: fffffffe8800504400
R13: 00000000000000004 R14: ffff88803dc32738 R15: 00000000000000000
FS: 0000000000000000(0000) GS:ffff88803dc00000(0000) knlGS:00000000000000000
CS: 0010 DS: 0000 ES: 0000 CR0: 00000000080050033
CR2: 00000000000515dc0 CR3: 0000000009ab8006 CR4: 00000000000370ef0
Call Trace:
<TASK>
? __die_body.cold+0x1a/0x1f
? die_addr+0x38/0x60
? exc_general_protection+0x14c/0x320
? asm_exc_general_protection+0x22/0x30
? rcu_core+0x2ca/0x7a0
? rcu_core+0x270/0x7a0
```



What about remote?

```
[+] setup user namespace  
[+] pin cpu @ core 0  
[+] pin cpu @ core 1  
[+] setup master conntrack  
[*] loop create conntrack expects...  
[*] trigger out-of-bounds decrement...  
[-] failed to add log expression
```




```
hexrabbit:mitigation-v3-6.1.55/ $ gdb vmlinux
GNU gdb (GDB) 15.0.50.20240328-git

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Loading GEF...
GEF is ready, type 'gef' to start, 'gef config' to configure
294 commands loaded for GDB 15.0.50.20240328-git using Python engine 3.11
[+] Not found /home/hexrabbit/.gef.rc, GEF uses default settings.
Reading symbols from vmlinux...

gef> b nf_tables_delrule
Breakpoint 1 at 0xffffffff81e56f20: file net/netfilter/nf_tables_api.c, line 3848.
gef> b nft_log_destroy
Function "nft_log_destroy" not defined.
```

```
hexrabbit:~/ $ ./scripts/diffconfig .config official.config
```

```
...
```

```
NFT_CONNLIMIT y -> n
```

```
NFT_HASH y -> n
```

```
NFT_LIMIT y -> n
```

```
NFT_LOG y -> n
```

```
NFT_MASQ y -> n
```

```
NFT_NAT y -> n
```

```
NFT_NUMGEN y -> n
```

```
NFT_OBJREF y -> n
```

```
NFT_OSF y -> n
```

```
NFT_QUEUE y -> n
```

```
NFT_QUOTA y -> n
```

```
NFT_REDIR y -> n
```

```
NFT_REJECT y -> n
```

```
NFT_SOCKET y -> n
```

```
NFT_SYNPROXY y -> n
```

```
NFT_TPROXY y -> n
```

```
NFT_TUNNEL y -> n
```

```
NFT_XFRM y -> n
```

```
...
```

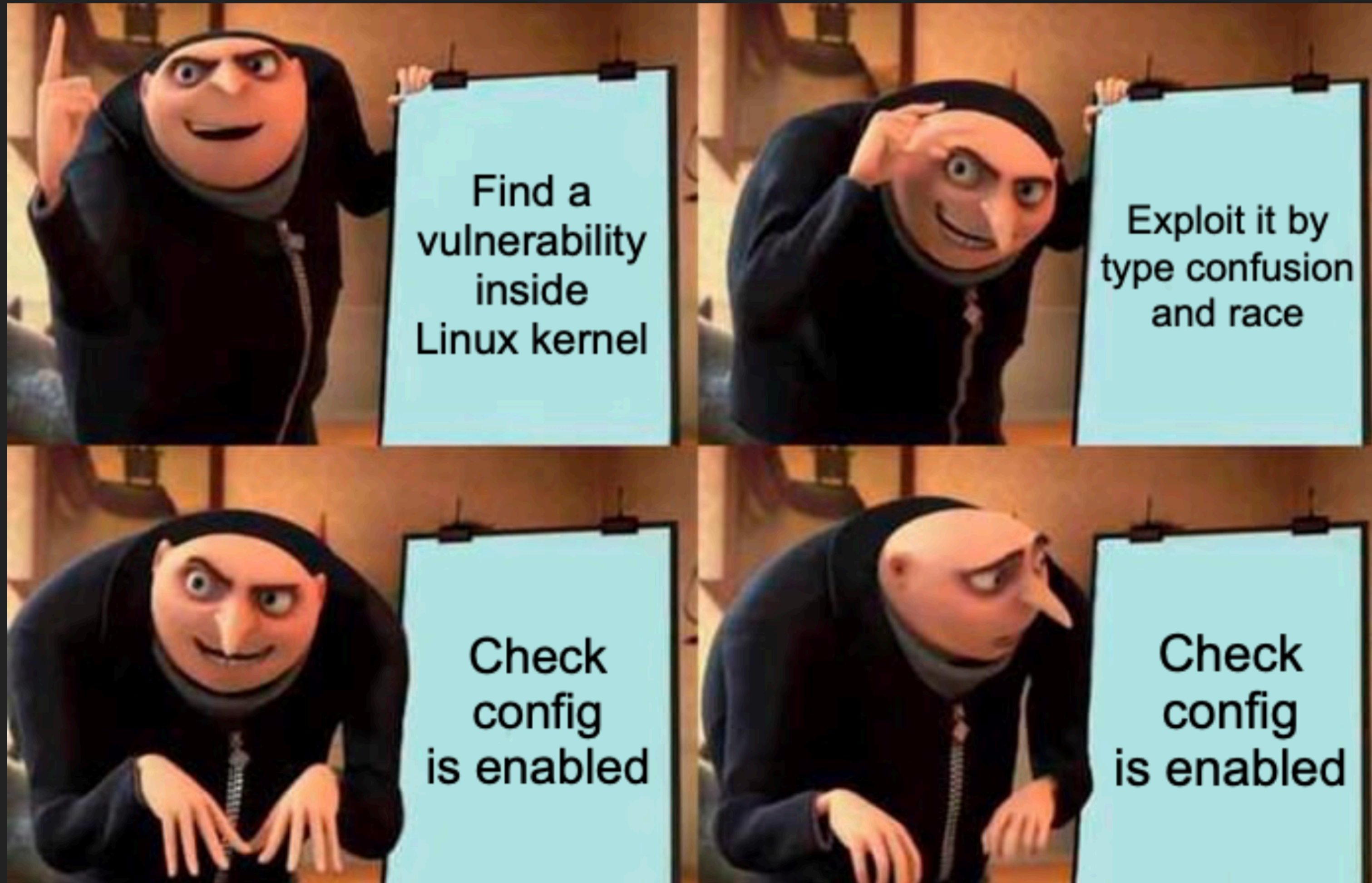
???

config is generated by the
official build script!



Mistakes

- Mitigation config = COS kernel config + custom security patch
 - Official instance use COS config version 6.1.55
 - nftables expression configs (NFT_*) enabled after 6.1.64
 - **Build script always fetch the newest COS config**



The third bug

Revisit the first bug

- The first bug was introduced by a patch set

netfilter: nf_tables: adapt set backend to use GC transaction API

Use the GC transaction API to replace the old and buggy gc API and the busy mark approach.

No set elements are removed from async garbage collection anymore, instead the `_DEAD` bit is set on so the set element is not visible from lookup path anymore. Async GC enqueues transaction work that might be aborted and retried later.

rbtree and pipapo set backends does not set on the `_DEAD` bit from the sync GC path since this runs in control plane path where mutex is held. In this case, set elements are deactivated, removed and then released via RCU callback, sync GC never fails.

Set garbage collection

- Some set implementation (e.g. rhash) support element timeout
- Two types of GC
 - Synchronous
 - **Asynchronous**

Revisit the first bug

- The first bug was introduced by a patch

What's the difference between new and old GC transaction API?

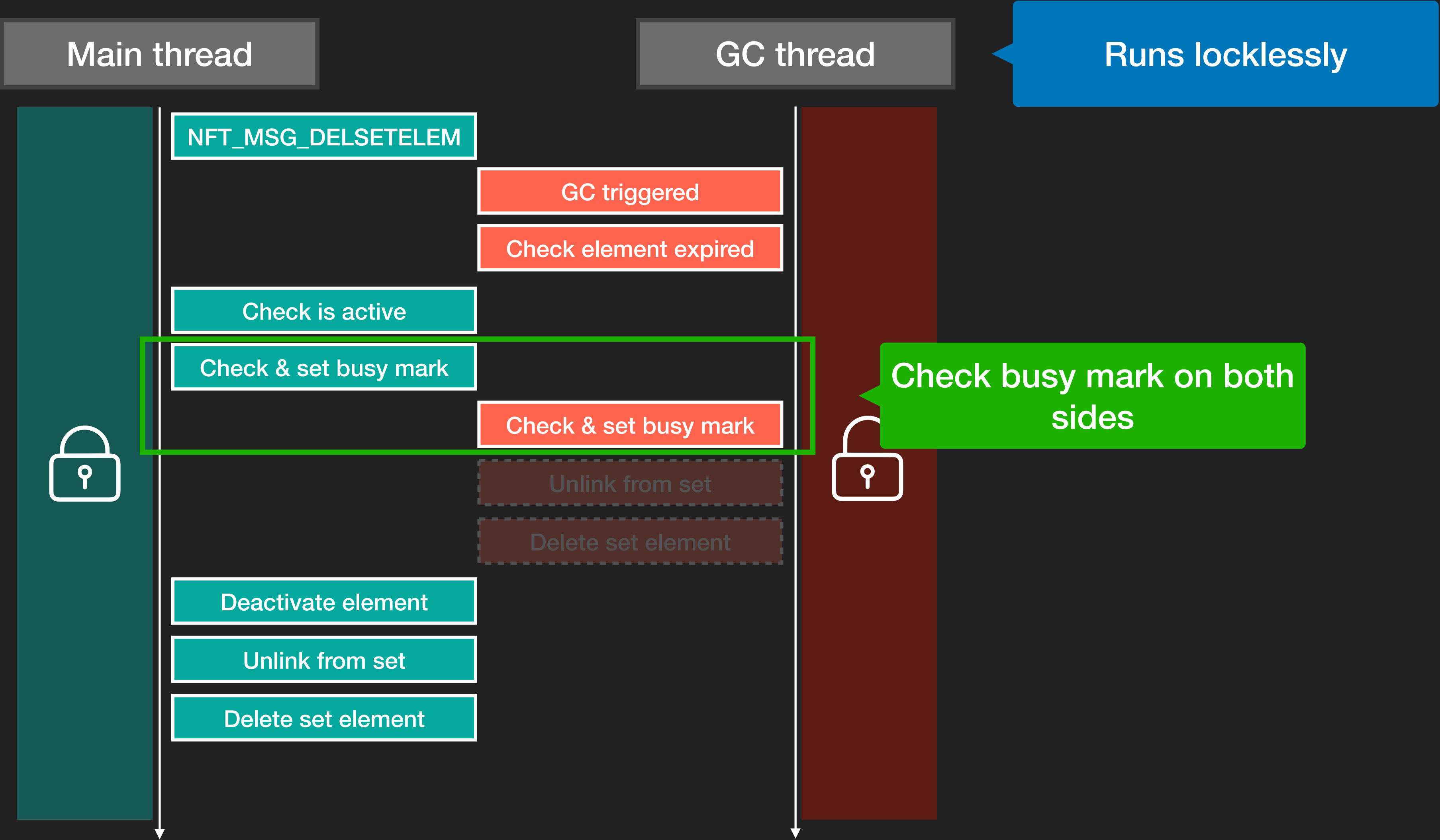
netfilter: nf_tables: adapt set backend to use GC transaction API

Use the GC transaction API to replace the old and buggy gc API and the busy mark approach.

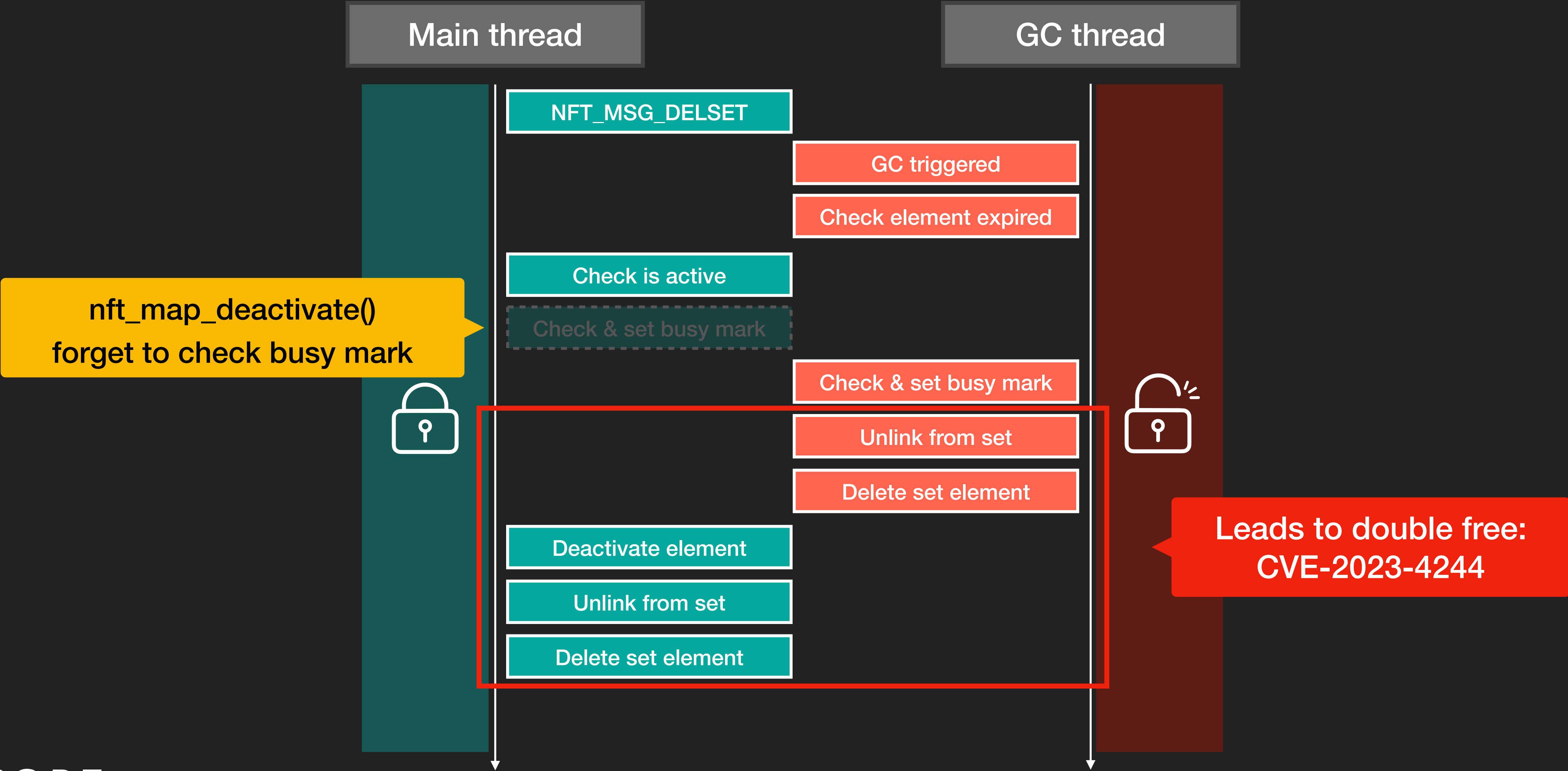
No set elements are removed from async garbage collection anymore, instead the `_DEAD` bit is set on so the set element is not visible from lookup path anymore. Async GC enqueues transaction work that might be aborted and retried later.

rbtree and pipapo set backends does not set on the `_DEAD` bit from the sync GC path since this runs in control plane path where mutex is held. In this case, set elements are deactivated, removed and then released via RCU callback, sync GC never fails.

Old GC design

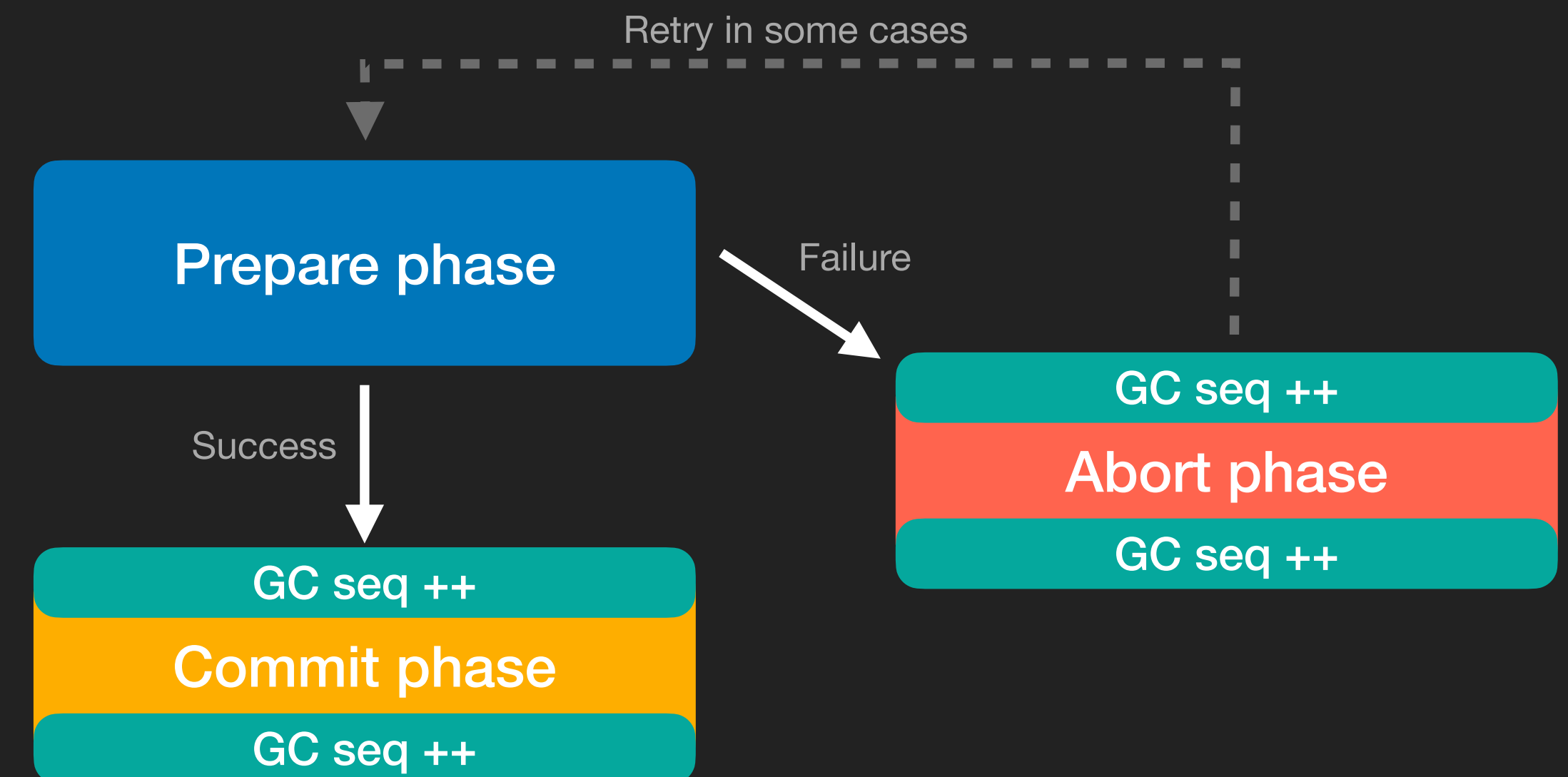


Old GC design

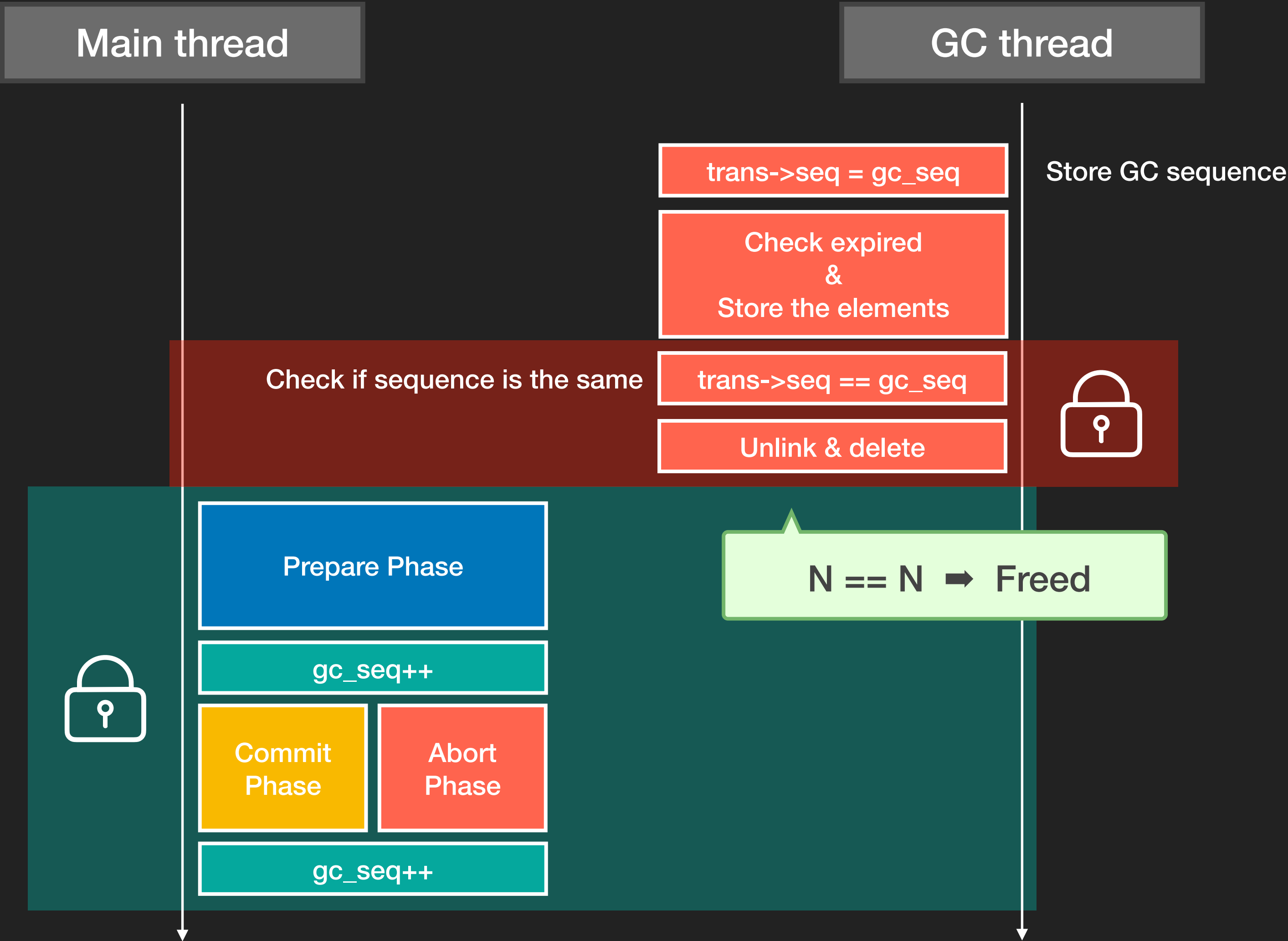


New GC design

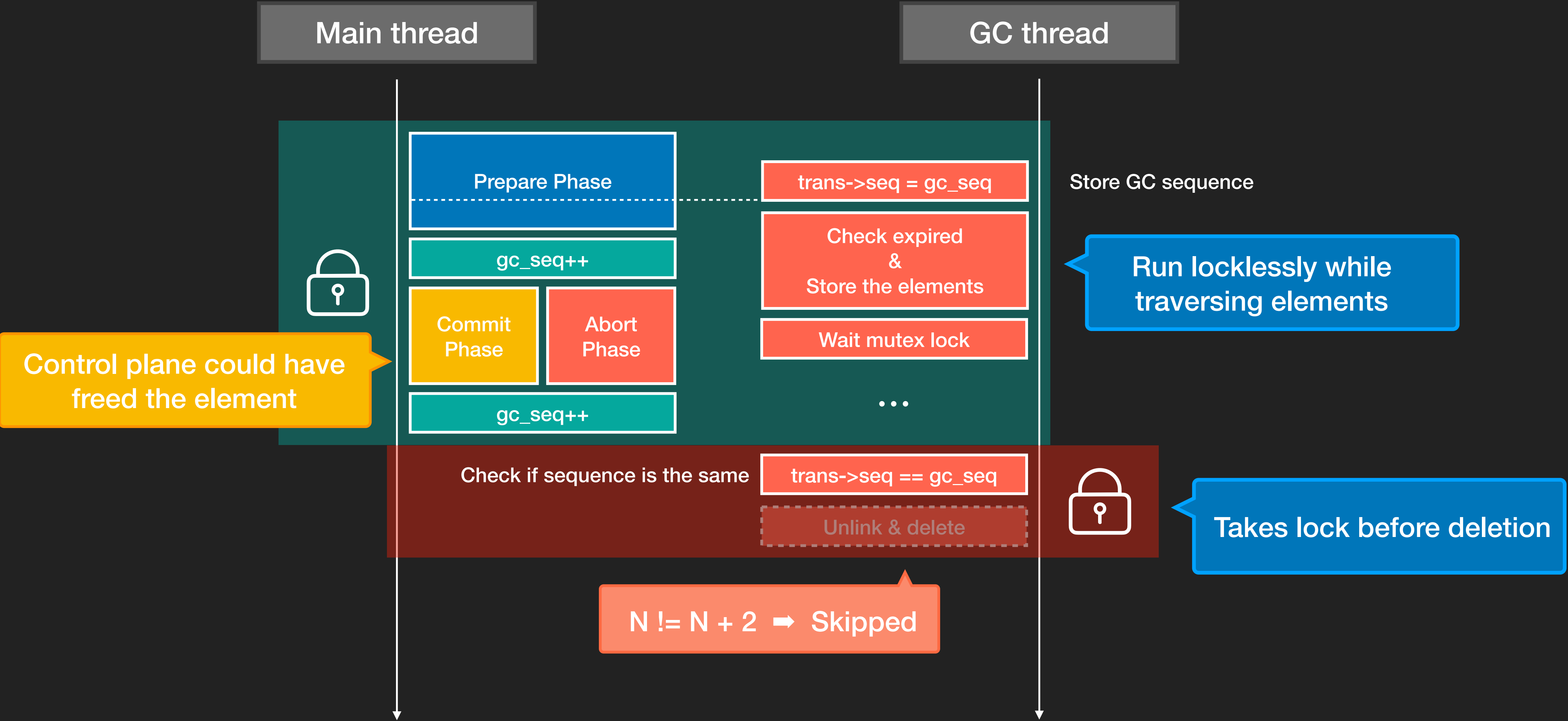
- Introduce “GC sequence”
 - Increment before & after modification to control plane
 - GC thread record & check it to prevent race
 - Acquire lock before check



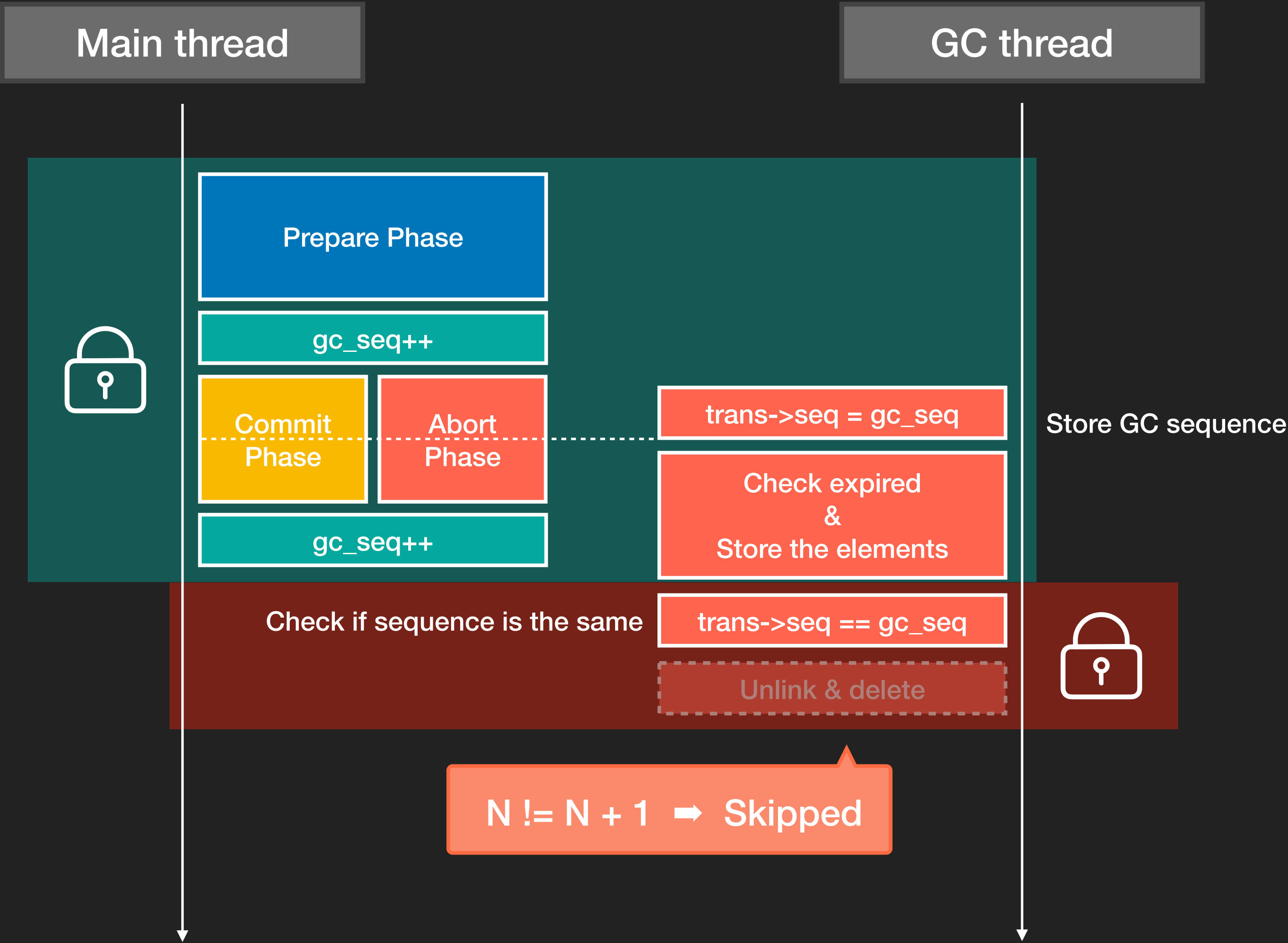
New GC design



New GC design



New GC design

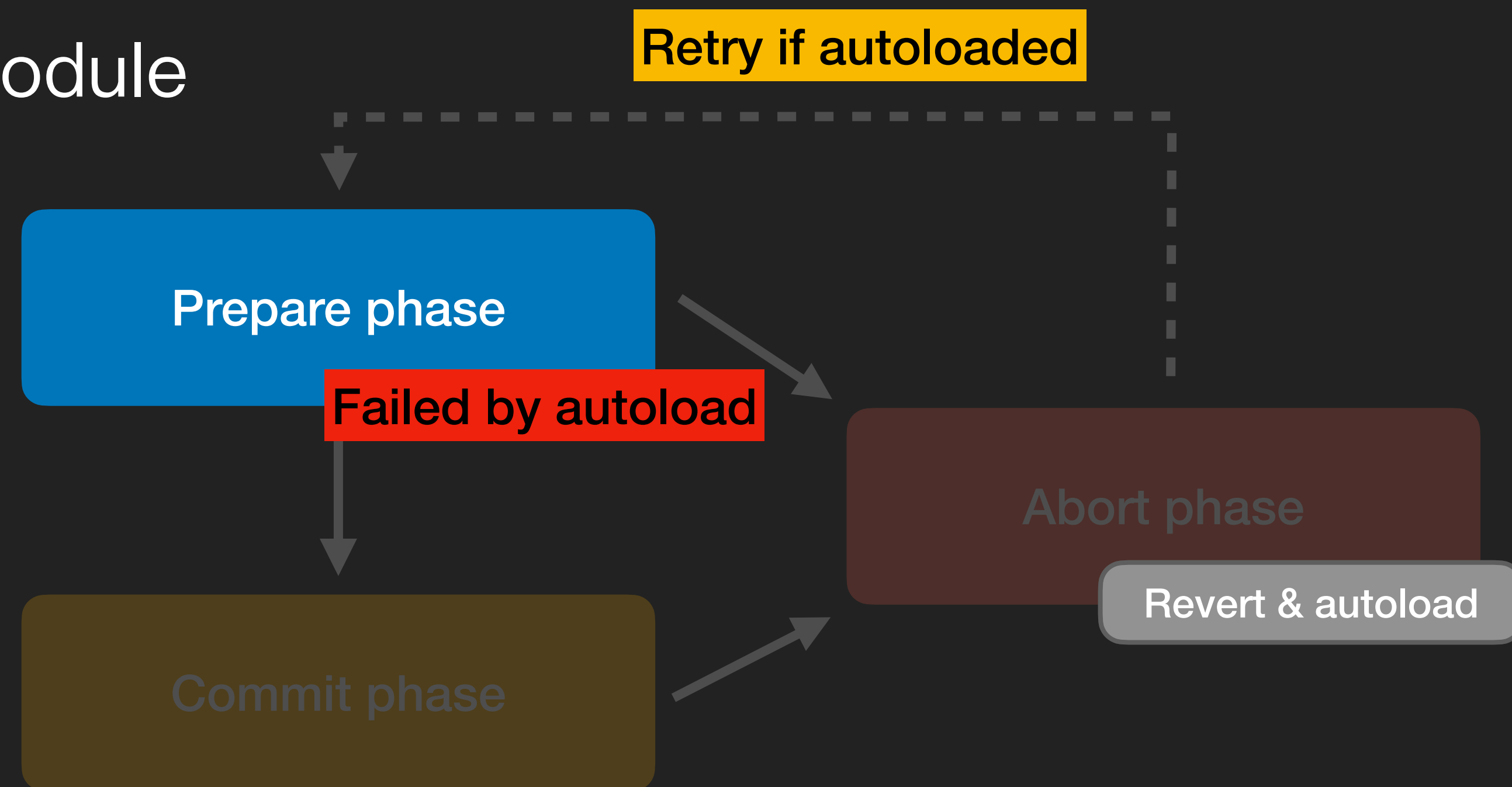


Always Locked = No Race?

not A

Module autoload

- If expression type is not found during type resolution
 - Instantly abort
 - Try to load corresponded type module
 - Retry batch request

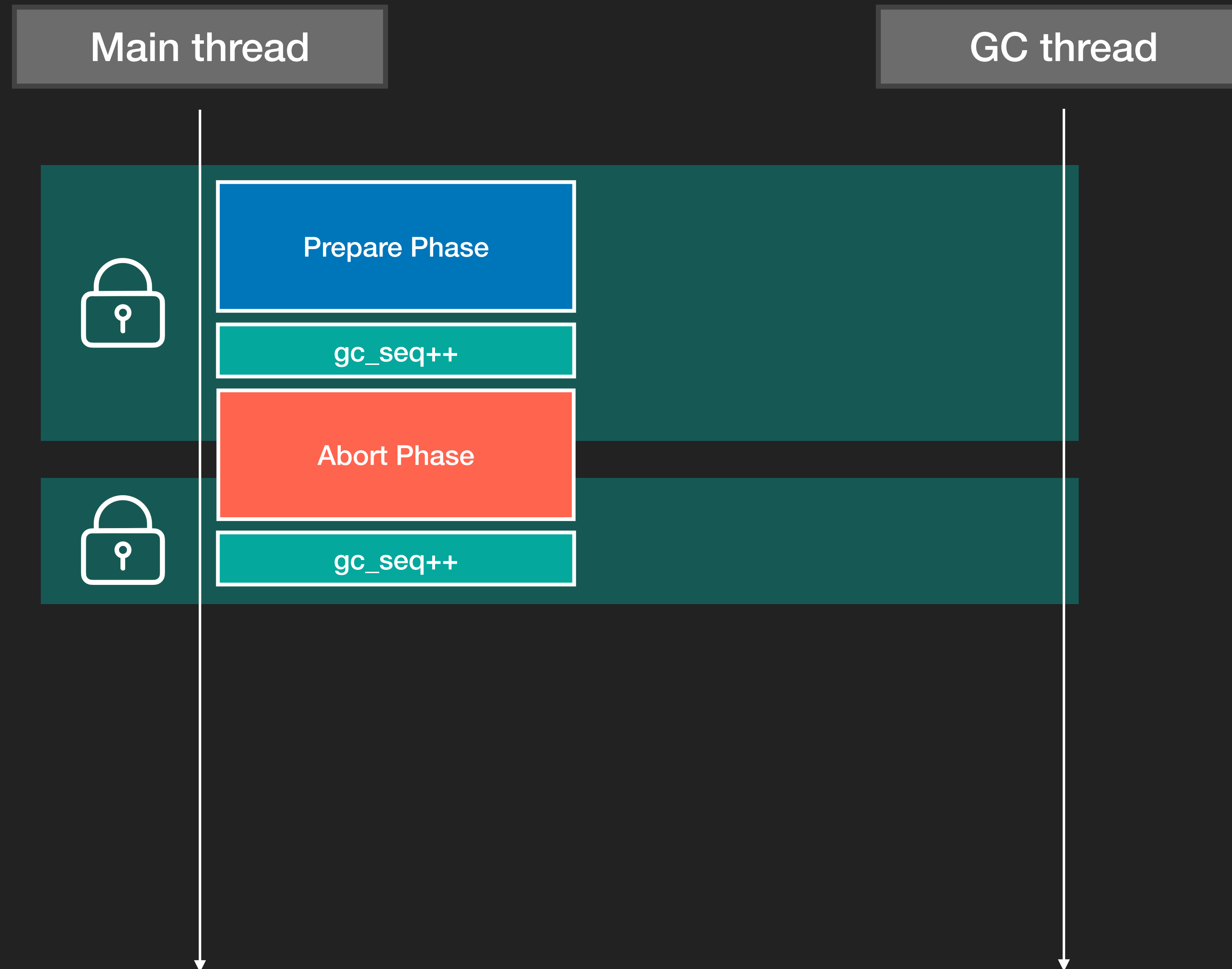


Module autoload

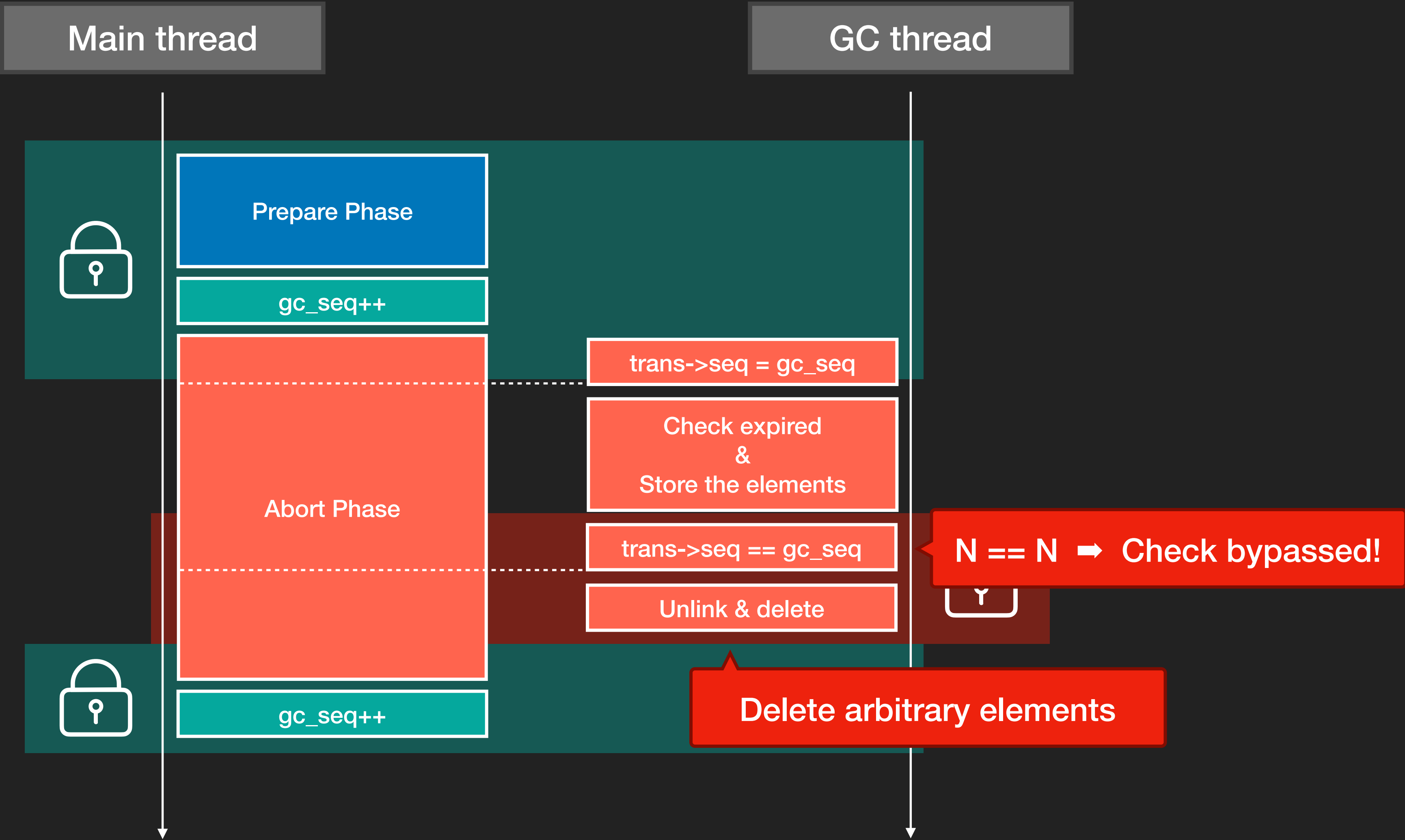
```
static void nf_tables_module_autoload(struct net *net)
{
    struct nftables_pernet *nft_net = nft_pernet(net);
    struct nft_module_request *req, *next;
    LIST_HEAD(module_list);

    list_splice_init(&nft_net->module_list, &module_list);
    mutex_unlock(&nft_net->commit_mutex);
    list_for_each_entry_safe(req, next, &module_list, list) {
        request_module("%s", req->module);
        req->done = true;
    }
    mutex_lock(&nft_net->commit_mutex);
    list_splice(&module_list, &nft_net->module_list);
}
```


Module autoload



Module autoload

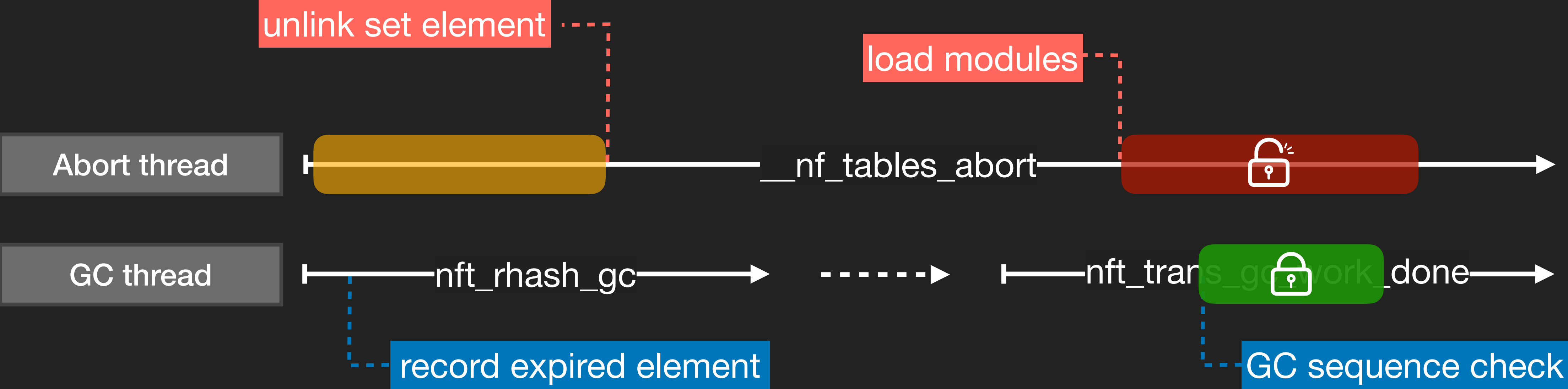


How to exploit the race?

How to exploit the race?

- Add a set element with a short timeout
 - Trigger autoload abort to revert set element add (unlink & kfree)
 - Before unlink, element timeout and collected by GC thread
 - Mutex unlocked during module autoload
 - GC thread grab mutex lock, bypassing GC sequence check
- ➔ Set element kfree'd 2nd time

Enlarge time windows

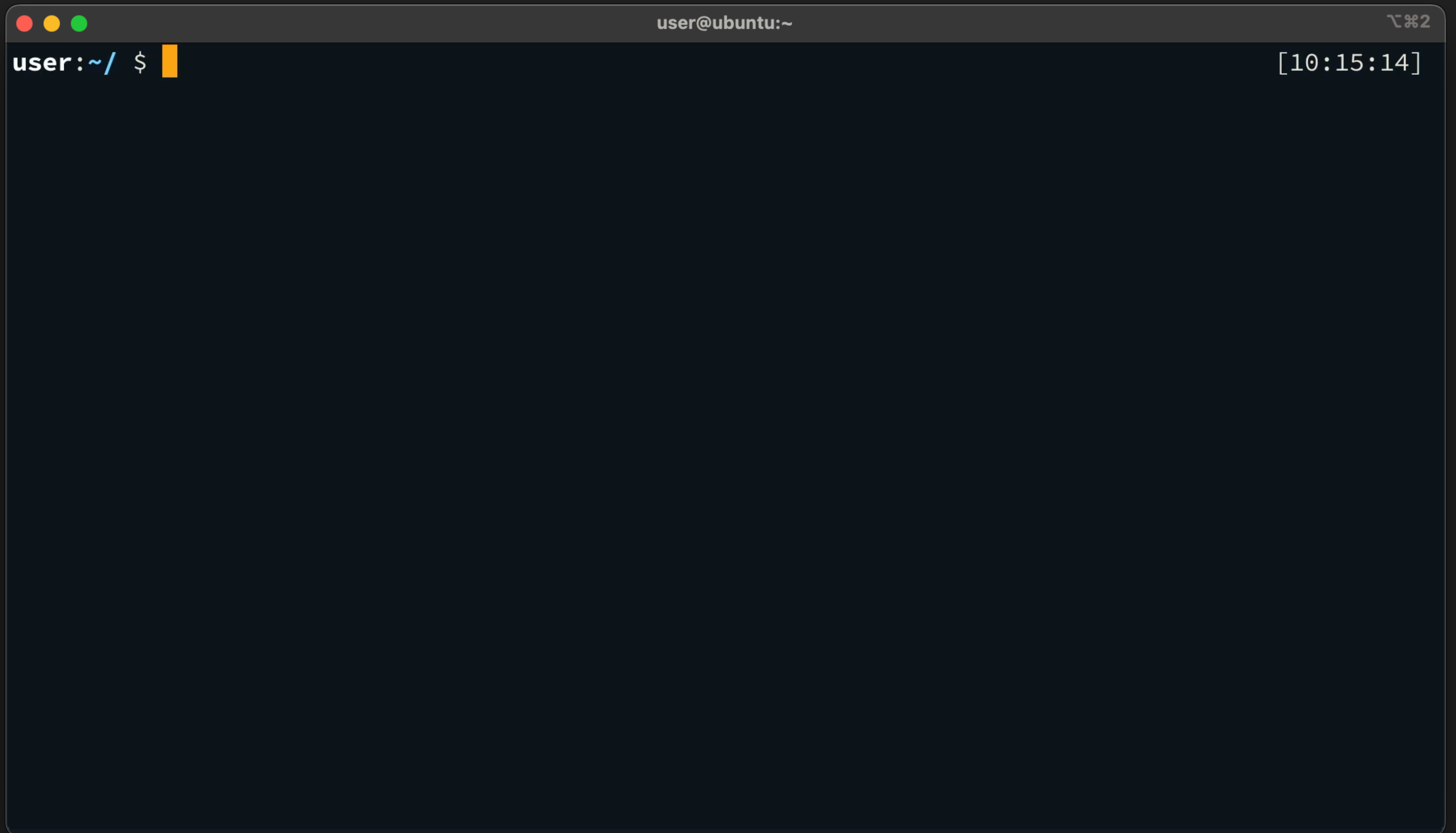


Race windows:

Enlarge time windows

-  From `__nf_tables_abort()` start to element being unlinked
 - ➔ Add a bunch of requests after add set element request (revert backwards)
-  Time span of mutex lock being released
 - ➔ Request multiple unknown type to trigger multiple module autoload

Demo



Submit to kernelCTF 🎉

Duplicate submission



exp156	2024-03-22T12:01:11.757Z	kernelCTF{v1:cos-105-17412.294.36:1711108845}	0-day		(dupe)
exp155	2024-03-22T12:00:48.331Z	kernelCTF{v1:cos-105-17412.294.36:1711108811}	1-day		(dupe)
exp154	2024-03-22T12:00:35.694Z	kernelCTF{v1:cos-105-17412.294.36:1711108805}	1-day		cos-105-17412.294.36
exp153	2024-03-22T12:00:35.074Z	kernelCTF{v1:its-6.1.81:1711108805}	0-day	(dupe)	
exp152	2024-03-22T12:00:26.934Z	kernelCTF{v1:its-6.1.81:1711108810}	1-day	its-6.1.81	

Duplicate submission

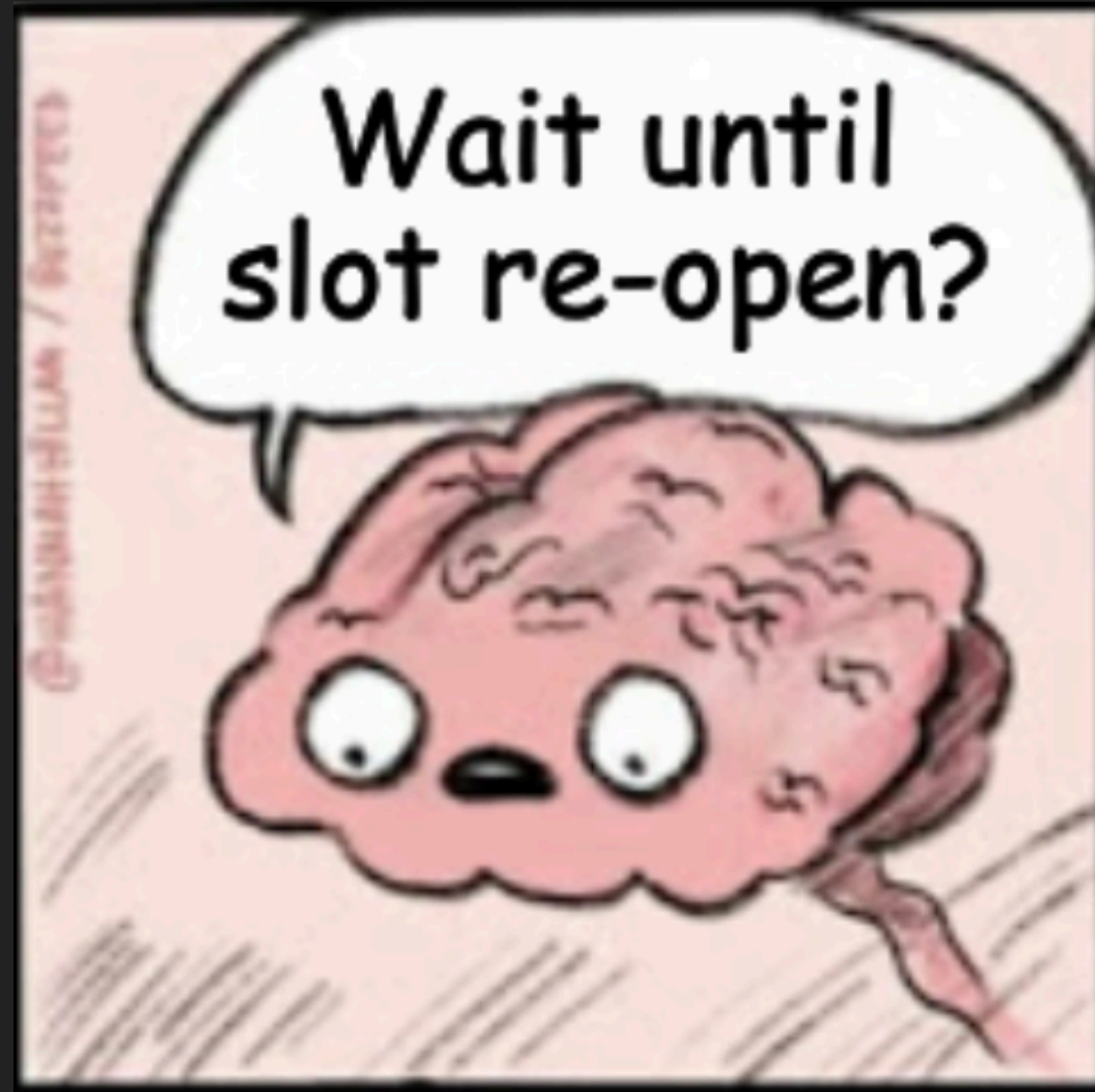
exp156	2024-03-27T12:00:35.000Z	CTF{v1:cos-105-17412.294.36:1711108845}	0-day		(dupe)
exp155	2024-03-27T12:00:35.000Z	CTF{v1:cos-105-17412.294.36:1711108811}	1-day		(dupe)
exp154	2024-03-27T12:00:35.000Z	CTF{v1:cos-105-17412.294.36:1711108845}	0-day		cos-105-17412.294.36
exp153	2024-03-27T12:00:35.000Z	CTF{v1:cos-105-17412.294.36:1711108811}	1-day		(dupe)
exp152	2024-03-27T12:00:26.900Z	CTF{v1:cos-105-17412.294.36:1711108845}	0-day	Its-6.1.81	

My submission

Someone beat me by
9 seconds



No problem,
Just wait for next slot ...?



kernelCTF announcement



KT 2024/02/09 21:04

Hey! We are planning to switch to LTS 6.6 (with nftables disabled), COS 109 (v6.1) and COS 105 (v5.15) from March 15th:
<https://github.com/google/security-research/commit/7632e6958a5d3eb2decfbd89a64324848b683e60>

The reason behind not including nftables in LTS releases, that it only enabled on COS / GKE (non-AutoPilot), not on upstream, on Android or ChromeOS, etc.

We already got 38 submissions for nftables (based on our public spreadsheet) during the last ~8 months and we've lost confidence that fixing the vulns one-by-one (and meanwhile hopefully not introducing new ones) actually makes this subsystem trustworthy and secure in the long run.



KT 2024/02/28 04:08

Hey! We've moved our planned date to switch to LTS 6.6 (with nftables disabled), COS 109 (v6.1) and COS 105 (v5.15) from March 15th to April 1st.

2024/03/22



kernelCTF opens slots
every 2 weeks avg



Still no bounty



Hope



A few days after...

Plot twist

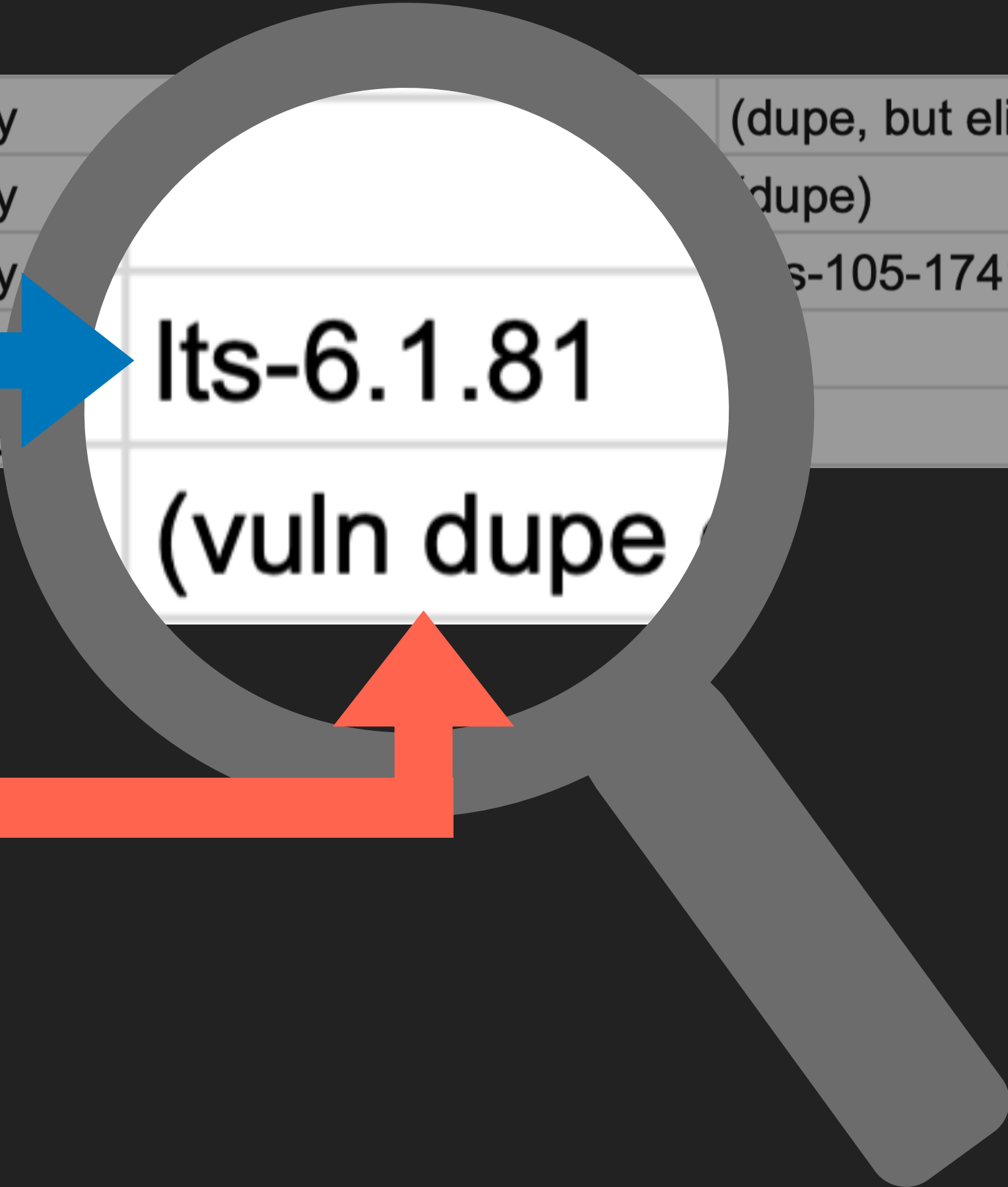


exp156	2024-03-22T12:01:11.757Z	kernelCTF{v1:cos-105-17412.294.36:1711108845}	0-day		(dupe)
exp155	2024-03-22T12:00:48.331Z	kernelCTF{v1:cos-105-17412.294.36:1711108811}	1-day		(dupe)
exp154	2024-03-22T12:00:35.694Z	kernelCTF{v1:cos-105-17412.294.36:1711108805}	1-day		cos-105-17412.294.36
exp153	2024-03-22T12:00:35.074Z	kernelCTF{v1:lts-6.1.81:1711108805}	0-day	(dupe)	
exp152	2024-03-22T12:00:26.934Z	kernelCTF{v1:lts-6.1.81:1711108810}	1-day	lts-6.1.81	

Plot twist

exp156	2024-03-22T12:01:11.757Z	kernelCTF{v1:cos-105-17412.294.36:1711108845}	0-day	(dupe, but eligible because)
exp155	2024-03-22T12:00:48.331Z	kernelCTF{v1:cos-105-17412.294.36:1711108811}	1-day	(dupe)
exp154	2024-03-22T12:00:35.694Z	kernelCTF{v1:cos-105-17412.294.36:1711108805}	1-day	(dupe, but eligible because)
exp153	2024-03-22T12:00:35.074Z	kernelCTF{v1:cos-105-17412.294.36:1711108805}	1-day	(dupe, but eligible because)
exp152	2024-03-22T12:00:26.934Z	kernelCTF{v1:cos-105-17412.294.36:1711108805}	1-day	(dupe, but eligible because)

My submission



Its-6.1.81
(vuln dupe)

Duplicate with the old submission

■ ACCEPTED 24.07.2024

kernelCTF exp153: Locking issue leading to Double Free in nf_tables affecting COS 5.15 and Linux 6.1

Google VRP - issu.ee/333000131



■ TRIAGED
22:36 | 08.04.2024

■ ACCEPTED
22:58 | 08.04.2024

REWARD DECIDED
\$61,837

Appeal reward !



Thanks!



Q&A

